SeatPlan: project report

Course: Introduction to Programming, Nova SBE, 2026 **Team:** Sergio and Martim (restaurants), Simao and Federico (events) **Version of the app described:** 0.1.0 (from `backend/app/config.py`) **Date of this report:** 10 November 2026

This report describes what SeatPlan is, why it is built the way it is, and how well it works. Every number about the app comes from the code or from a command run on 10 November 2026; the command is named next to the number. Things that have not happened yet (a real deployment on Vercel, real email through Resend, usability sessions with real people, the presentation) are written as "not done yet". Where another document in the repository already explains something in full, this report gives the short version and links to it.

Contents

1. Summary
2. The problem and the users
3. Requirements
4. Design decisions (decision records DR-01 to DR-22)
5. Architecture and data model
6. How double booking is made impossible
7. The optimisers
8. Real-time design
9. Email
10. Security and privacy
11. Testing, with real results
12. User experience and accessibility
13. Deployment
14. Limits and what is not done
15. Lessons
16. Traceability matrix
17. References

Appendix A lists every command run for this report and its real output.

1. Summary

SeatPlan is a web app that lets people choose their own seat or table on a live two-dimensional map. It has two "verticals" (two kinds of customer) that share one core:

- **Events.** A guest at a wedding or a conference opens a personal link, says whether they are coming, says who they know, sees who sits where (names are shortened or hidden according to the event's privacy rules) and picks a seat. The organiser sees the full map, lets the computer fill the seats of people who did not choose, and can re-plan after changes while moving as few people as possible.
- **Restaurants.** A diner picks a date, a party size, a time and a table, or lets SeatPlan choose. Staff see a live floor plan, seat walk-ins, move or join tables, ask "what if a party of six walks in now?" and read a dashboard.

Three ideas carry the whole design:

1. **The database decides who gets a seat.** Every seat, table slot and temporary hold is protected by a UNIQUE constraint (or a primary key) in the database. Two people can press "confirm" at the same instant on two different servers and exactly one of them wins. The others get a polite "this seat was just taken" message. Section 6 gives the argument and the tests that check it.
2. **The optimiser explains itself and never overrides a person.** Seat and table suggestions come with reasons in plain words ("Next to 2 people you know", "Exact fit for 4"). Choices made by guests, diners or staff are passed to the optimiser as fixed and are never moved. Results are repeatable: the same data and the same seed give the same plan. The quality of the fast method is measured against a proven optimum on small cases (section 7).
3. **It is built for serverless hosting.** The planned host is one Vercel project: a static React frontend plus one Python function. Nothing relies on memory shared between requests. Live updates use a version counter in the database that the browser watches through a short event stream, or by polling every 2 seconds when the stream fails (section 8). Emails are written to an outbox table and sent later by a scheduled job (section 9).

Size of the work (counted on 10 November 2026 with `find` and `wc -l`): 83 Python files with 11,917 lines in `backend/app`; 41 backend test files with 6,889 lines; 122 TypeScript and TSX files with 11,756 lines in `frontend/src` (files named `*.test.*` not counted); 3,196 lines of CSS (recounted in the final verification, after its small frontend fixes). The API has 56 paths and 65 operations (counted from `docs/openapi.json`, which the architecture work exported from the real app today). The database has 24 tables (counted from the demo database seeded today).

Results we measured today (final verification run, after the last code change of the day; the same numbers as `README.md` and `docs/test-report.md`):

Check	Result (real output, 10 November 2026)
Backend tests	`469 passed, 1 warning in 31.52s`

Check	Result (real output, 10 November 2026)
Backend tests on Postgres 16.2	`468 passed, 1 skipped, 1 warning in 62.08s (0:01:02)`
Concurrency tests only	`4 passed, 465 deselected, 1 warning in 2.14s`
Property-based tests only	`12 passed, 457 deselected, 1 warning in 1.76s`
Lint and format	`All checks passed!` and `138 files already formatted`
Frontend unit tests	`Test Files 15 passed (15)`, `Tests 134 passed (134)`
Events optimiser gap	20 of 21 small instances at the proven optimum; one 3.7 % below
Restaurant optimiser gap	19 of 20 small services fully optimal; 4 of 8 demo-sized services reached the optimal covers, the other 4 were 1.3 % to 3.6 % below
Browser tests (`npm run e2e`)	`12 passed (10.8s)`, `12 passed (10.4s)`, `12 passed (9.5s)`

What is not done yet: a real Vercel deployment, real email delivery, any usability session with real people, screen-reader and real-device testing, and the presentation. Section 14 lists every known limit.

2. The problem and the users

2.1 The problem

Seating is a small problem that people care about a lot. At a wedding, the couple spends evenings moving paper names around a table plan, and guests still end up next to strangers. At a conference, people who came together sit apart because the seats were filled in arrival order. In a restaurant, the booking system gives a time but not a table, so the couple who asked for the window table finds out at the door whether they got it, and a party of two may be put at the only table for six.

Existing tools usually solve one half of this. Either the organiser plans everything alone (the guests have no say), or guests pick freely (and nobody checks that families sit together or that the accessible seats stay free for the people who need them). SeatPlan tries to do both: people choose, and an optimiser fills the gaps around their choices without breaking any of them.

There is also a technical problem hidden inside. When people choose for themselves, two of them will sooner or later try to take the same seat at the same moment. A system that "checks if the seat is free, then saves" can give the seat to both, because both checks happen before either save. This is the classic double-booking bug, and it gets more likely, not less, when the app runs on many serverless instances at once. Section 6 is about how we closed it.

2.2 The users

The product specification (`docs/product-spec.md`) names four kinds of user. Each has a persona in `docs/personas.md`. The personas are assumptions written by the team, not the result of user research; no interviews were done.

User	Persona (fake)	What they want	Main screens
Event guest (attendee)	Rita	Open the link, say who she knows, sit near them, done in two minutes on a phone	`/e/<token>`: `frontend/src/pages/events/AttendeePage.tsx` and its steps
Event organiser	Joana	See who has not chosen, fill the rest, keep families together, print place cards	`/organiser/<event id>`: `OrganiserPage.tsx` with tabs for guests, optimise, print and settings
Restaurant diner	Tiago	Book a table for four on Friday, by the window if possible, and change it later	`/r/<venue id>`: `BookingPage.tsx`; `/b/<token>`: `ManageBookingPage.tsx`
Restaurant staff	Marta	See tonight's floor at a glance, seat walk-ins fast, not lose the big table to a couple	`/staff/<venue id>`: `StaffPage.tsx` with floor, dashboard, optimise and schedule

There is also a floor-plan editor (`/editor/<venue id>`, `frontend/src/pages/editor/`) for organisers and staff to draw rooms, tables and seats.

2.3 What success looks like

For each user, the product specification lists user stories with acceptance criteria that point to the code and the tests (28 stories: A1 to A7 for attendees, O1 to O10 for organisers, D1 to D5 for diners, S1 to S6 for staff). This report does not repeat them; section 16 maps the requirements to code, tests and documents instead.

3. Requirements

3.1 Where the requirements come from

The official course brief is not stored in this repository, so this report could not quote it word by word. The requirements below are taken from what the project itself agreed to build: `docs/CONTRACT.md` (the design agreement between all modules, sections 1 to 9), `README.md` and `docs/product-spec.md`. The team should paste the brief's exact criteria into the empty table at the end of section 16 before submission and check each one against this list.

3.2 Functional requirements

ID	Requirement	Source
R1	People choose their own seat or table on a live 2D map	Contract 1
R2	Two verticals (events, restaurants) share one core	Contract 1
R3	Event guests open a personal link and answer the invitation (RSVP)	Contract 1, 5
R4	Guests say who they know and get seat suggestions near those people, with reasons	Contract 1, 5
R5	Guests see who sits where, within the event's privacy rules	Contract 1, 5, 7
R6	The organiser sees the full map and who has not chosen yet	Contract 5
R7	The organiser adds guests one by one or by CSV, and exports them	Contract 5
R8	The optimiser auto-fills the people who did not choose	Contract 1, 6
R9	The optimiser re-plans with as few moves as possible	Contract 1, 6
R10	Scenarios can be saved, compared, applied, undone and redone; "what if" shows the effect of a move	Contract 5
R11	An exact check compares the heuristic with a proven optimum on small cases	Contract 5, 6
R12	A diner picks date, party size, time slot and table, or lets the app choose	Contract 1, 5
R13	A diner gets a manage link to change or cancel; a waitlist exists when full	Contract 5
R14	Staff see a live floor, set booking status, move and combine tables	Contract 5
R15	Staff seat walk-ins and ask "what if" without changing anything	Contract 5
R16	Staff read a dashboard and edit the opening schedule	Contract 5
R17	Staff can preview and apply an optimised plan for a service	Contract 5, 6
R18	Organisers and staff can draw the floor plan	Product spec 3.1
R19	Emails confirm, change, cancel and remind, with a calendar invite	Contract 2, 4, 5

3.3 Rules that must always hold (hard requirements)

ID	Requirement	Source
H1	No double booking, ever: the database decides	Contract 4
H2	A seat or table can be held for 5 minutes before confirming	Contract 4

ID	Requirement	Source
H3	Human choices and hard rules (one person per seat, accessibility, reserved seats, blocked seats, table sizes) are never broken by the optimiser	Contract 1, 6
H4	Same problem and same seed give the same result; the iteration budget is the stopping rule	Contract 6
H5	Other attendees never receive email, dietary or accessibility data; names pass through the privacy rule	Contract 7

3.4 Non-functional requirements

ID	Requirement	Source
N1	Runs on Vercel: static frontend plus one Python function; no shared memory, no long-lived sockets	Contract 2
N2	Live updates that work on serverless	Contract 2
N3	Security basics: secrets from the environment, bound SQL parameters, validated bodies, rate limits, hashed tokens, CORS, no raw HTML	Contract 7
N4	Accessible: keyboard use of the map, states not shown by colour alone, enough contrast	Contract 2, product spec
N5	Tested with standard tools (pytest, Hypothesis, vitest) and end to end	Contract 2
N6	Code a beginner can explain in a live walkthrough: small modules, type hints, comments that say why, a `READ_ME_FIRST.md` per package	Contract 8
N7	Fake people only in data and tests	Contract 7
N8	Few dependencies; standard library first	Contract 2

4. Design decisions

Each record below follows the same four parts: the **context** (what problem we faced), the **options** we considered, the **choice**, and the **consequence** (what we gained and what it cost). They are drawn from `docs/decisions-log.md` and from the code. The log has the full list of deviations from the contract, with dates; these records are the ones that shaped the system most.

DR-01. Let the database decide who gets a seat

- **Context.** Two people can confirm the same seat at the same moment, on two different serverless instances that share nothing but the database.
- **Options.** (a) Check in Python whether the seat is free, then insert. (b) Use a lock in the server's memory. (c) Put a UNIQUE constraint on the table and treat a constraint error as "taken".
- **Choice.** (c). `seat_assignments` has `UNIQUE(event_id, seat_id)` and `UNIQUE(event_id, guest_id)`; `table_slots` has the primary key `(table_id, date, slot_index)`; `holds` has `UNIQUE(scope, resource_kind, resource_id, slot_key)` (`backend/app/models.py`).
- **Consequence.** Option (a) has a gap between the check and the insert; option (b) does not work across instances. With (c) correctness does not depend on our Python code being right, only on the database doing what its documentation promises. The cost is that every write path must catch `IntegrityError` and turn it into a 409 answer; `core/holds.py` does this in one helper, `commit_or_seat_taken`. Section 6 has the full argument.

DR-02. Five-minute holds, also enforced by the database

- **Context.** Choosing a seat takes a few seconds; filling in a restaurant booking form takes longer. Without a hold, someone could lose the table while typing their name.
- **Options.** (a) No holds; first to confirm wins. (b) A hold stored in the browser only. (c) A hold row in the database with an expiry time.
- **Choice.** (c), with `HOLD_MINUTES = 5` in `backend/app/core/holds.py`. Expired holds are deleted inside the same transaction that tries to take the resource, so no background job is needed.
- **Consequence.** The person who held first is protected for 5 minutes. Nobody else can hold the same seat, because the hold table has its own UNIQUE constraint. Closing the tab does not release a hold; it simply runs out after 5 minutes. Expired rows are not swept in bulk; they are replaced when someone tries to take the same resource.

DR-03. Restaurant tables are held and booked per 15-minute slot

- **Context.** Restaurant bookings overlap in time: a booking from 19:00 to 20:30 must block a booking from 20:00, but not one from 20:30.

- **Options.** (a) One row per booking with start and end times, plus an overlap check in Python. (b) A database exclusion constraint on time ranges (Postgres only). (c) Cut the day into 15-minute slots and store one row per table per slot.
- **Choice.** (c). A booking writes one `table_slots` row per table per slot it occupies; a hold writes one `holds` row per table per slot, with the slot index as `slot_key` (decisions log, "restaurants vertical API", item 2).
- **Consequence.** Overlap becomes a simple equality that a primary key can refuse, and it works the same on SQLite (tests) and Postgres (production). Option (b) would not run on SQLite. The cost is more rows (a 90-minute stay is 6 rows per table) and the rule that a stay cannot cross midnight (decisions log, item 9).

DR-04. Python, FastAPI and SQLAlchemy on the backend

- **Context.** The course language is Python, and every team member must be able to explain the backend.
- **Options.** (a) Django. (b) Flask. (c) FastAPI with Pydantic and SQLAlchemy 2.
- **Choice.** (c), with the synchronous SQLAlchemy API (contract section 2). FastAPI builds APIs from standard Python type hints [FASTAPI]; SQLAlchemy declares multi-column rules with `UniqueConstraint` [SQLA-CONSTR] and groups writes in session transactions that commit or roll back together [SQLA-TX]. Versions installed today: FastAPI 0.162.2, SQLAlchemy 2.1.8, Pydantic 2.14.3.
- **Consequence.** Typed request bodies are checked automatically (unknown fields are rejected), and the OpenAPI description comes for free, which the architecture work exported to `docs/openapi.json`. Synchronous database calls are easier to read than `async` ones; the one place where waiting matters (the live stream) is written as an `async` generator so it holds no worker thread (DR-12).

DR-05. SQLite locally and in tests, Postgres in production

- **Context.** Tests should be fast and need no server; production needs a database that many instances can share.
- **Options.** (a) Postgres everywhere. (b) SQLite everywhere. (c) The same SQLAlchemy code on both.
- **Choice.** (c). Each backend test gets its own SQLite file. An opt-in run on Postgres exists: with `SEATPLAN_TEST_POSTGRES` set, every test gets its own new Postgres database (`backend/tests/conftest.py`).
- **Consequence.** Tests run in about half a minute with no setup. The risk is that the two databases differ; the README records one full run of the suite on a local Postgres 16.2 (`430 passed, 1 skipped`), done before today. It was not repeated today, and CI uses SQLite only.

DR-06. No table creation when the app starts on Postgres

- **Context.** Several serverless instances starting at the same moment raced on `CREATE TABLE` and crashed (decisions log, "fixes after the adversarial review (platform)", item 5).

- **Options.** (a) Keep `create_all()` at start-up. (b) A migration tool such as Alembic. (c) A separate command run once by a person.
- **Choice.** (c): `python -m app.seed.migrate`, which takes a Postgres advisory lock (`pg_advisory_xact_lock` in `backend/app/db.py`) so it is safe to run twice or in parallel. On local SQLite the app still creates tables itself.
- **Consequence.** No more start-up races. The cost: there is no migration tool, so a new column on an existing table needs a hand-written `ALTER TABLE` or a fresh database.

DR-07. Public ids and hashed tokens instead of integer ids in URLs

- **Context.** URLs such as `/r/12` would let anyone count and visit every restaurant or booking.
- **Choice.** Integer keys stay inside the database. Anything in a URL is a random `public_id` or a secret token made with `secrets.token_urlsafe(32)`. Tokens are stored only as SHA-256 hashes (`backend/app/security.py`, `hash_token`). A wrong token and a revoked token give the same 404.
- **Consequence.** A leaked database does not leak working personal links. The cost: the server can never show a guest's link again, so a lost link must be replaced with "New link", which revokes the old one (decisions log, "events screens", item 2). It also means an email sent when an organiser acts cannot contain the guest's personal link (decisions log, "integration", item 3).

DR-08. Privacy of names in one function

- **Context.** Guests may see who sits where, but the organiser chooses how much: full names, first name and last initial, or no names. A guest may also ask to be anonymous.
- **Choice.** One function, `display_name()` in `backend/app/core/privacy.py`, through which every name sent to another attendee passes. The people search (`GET /a/{token}/people?q=`) searches only the name that attendee is allowed to see (decisions log, "events vertical API", item 2).
- **Consequence.** The rule lives in one place and is tested by `tests/test_privacy.py` and `tests/test_attendee_privacy.py`. The frontend repeats the rule only to preview a mode before saving.

DR-09. Recommendations rank by the guest's own list only

- **Context.** "Knowing" is treated as mutual in the optimiser's score. If a seat suggestion used that score, a guest could notice that the seat next to a person they never listed ranks first, and guess that this person listed them.
- **Options.** (a) Show the raw gain. (b) Rank by the mutual score but hide the number. (c) Rank using only the guest's own list and show a coarse label.
- **Choice.** (c). `events_service/recommend.py` removes the guest from every other guest's "knows" set before ranking; the API returns `rank` and `match` (`strong`, `some`, `none`), never the gain (decisions log, "fixes after the adversarial review", item 1).

- **Consequence.** The order cannot leak who listed whom. Test: `test_recommend_never_reveals_who_listed_the_guest` in `tests/test_optimiser_events_search.py`. The planner's own objective is unchanged.

DR-10. A pure-Python optimiser package with no database imports

- **Context.** The optimiser must be testable on its own and explainable.
- **Choice.** `backend/app/optimiser/` takes frozen dataclasses (`SeatSpec`, `GuestSpec`, `TableSpec`, `PartySpec`) and returns plain results. Routers and the service packages convert rows to these and back (contract section 6).
- **Consequence.** Optimiser tests need no database and run in milliseconds; the gap-report scripts reuse the same functions. The cost is the conversion code in `events_service/problem.py` and `restaurant_service/service_plan.py`.

DR-11. Repeatable results: one seeded generator, iteration budget as the stopping rule

- **Context.** If a plan depends on how fast the machine is, the same button gives different answers on a laptop and on Vercel, and tests become flaky.
- **Choice.** All randomness comes from one `random.Random(seed)` passed down explicitly. The search stops after `max_iterations` (20,000 for events, 5,000 for restaurants). A wall-clock limit of 15 seconds exists only as a safety cut-off (`SAFETY_TIME_LIMIT_S`).
- **Consequence.** Same data and same seed give the same plan (tests `test_solve_same_seed_same_result` and `test_same_seed_gives_the_same_plan`). We ran the events gap report twice today and the score columns were byte-for-byte identical (Appendix A). The 15 s cut-off was lowered from 60 s so that an answer always comes back before Vercel's 30 s `maxDuration`, even with an exact check in the same request; a test reads `vercel.json` and fails if the two stop fitting (`test_safety_cut_off_fires_before_the_platform_limit`).

DR-12. Live updates by version counter, short event stream and polling

- **Context.** Serverless functions cannot keep a WebSocket open for long and do not share memory, so a server cannot "push" a change it did not see.
- **Options.** (a) WebSockets. (b) A paid real-time service. (c) A version number per scope in the database, sent through a short server-sent event stream, with polling as the fallback.
- **Choice.** (c). Every write bumps `live_versions` for its scope in the same transaction. `GET /live/{scope}/stream` checks the counter about once a second and closes after about 25 s; the browser reconnects. If the stream fails or is refused, the page polls `GET /live/{scope}/version` every 2 s.
- **Consequence.** Only a number travels; the page fetches the data again, through the same endpoints and privacy rules as before. It works on any host. The cost is up to one or two seconds of delay and one database read per open stream per second (section 8).

DR-13. An email outbox instead of sending inside the request

- **Context.** A booking must not fail or hang because an email service is slow, and the same email must not be sent twice.
- **Choice.** `enqueue()` inserts a row into `email_outbox` in the same transaction as the booking; a scheduled call to `POST /api/internal/email/process` sends due rows. The key `idempotency_key` is UNIQUE. A row is claimed by one conditional UPDATE (status `sending`, a 10-minute lease), so two overlapping runs never send the same email twice (decisions log, "email system", item 1).
- **Consequence.** A rolled-back booking leaves no email; failures are retried with growing waits (5, 10, 20, 40 minutes) and stop after the fifth. The cost is delay: emails go out at the next scheduled run (every 15 minutes in `.github/workflows/email-cron.yml`), not at once.

DR-14. An allowlist for every email provider

- **Context.** A demo must never email a real stranger by mistake.
- **Choice.** Only addresses in `EMAIL_ALLOWLIST` ever reach a provider; every other row is marked `skipped`. An empty list means nobody gets email. The rule applies to the console provider too (decisions log, "email system", item 2).
- **Consequence.** One rule with no exceptions is easy to trust. Locally, the demo addresses must be added to the allowlist to see them in the console.

DR-15. The organiser's history protects guests' own choices

- **Context.** Auto-fill, re-plan, undo and redo all rewrite many seats at once. A guest who picked their own seat must keep it.
- **Choice.** "Human choices" means sources `self` (the guest) and `organiser`. Both are passed to the optimiser as fixed, and applying a scenario never overwrites either. Undo and redo may restore organiser moves, but never undo or re-create a guest's own choice (decisions log, "events vertical API", item 5).
- **Consequence.** H3 holds even across history operations. Scenario application and undo do not send seat emails (to avoid mailing everyone at once), but each guest's single reminder is kept in step.

DR-16. One lock order per event, and "please try again" for deadlocks

- **Context.** On Postgres, two transactions that lock the same rows in a different order can deadlock; Postgres then aborts one of them. Before the fix, two guests racing for one seat (one confirming, one holding) could produce a deadlock and a 500 error (README, "Test report").
- **Choice.** Every write on an event first calls `seating.lock_event`, which bumps the event's live counter: one row update that every event transaction makes first. Any remaining deadlock or serialization failure (SQLSTATE `40P01` or `40001`, `_RETRY_SQLSTATES` in `backend/app/main.py`) is answered with 409 `conflict` "please try again" instead of a 500.

- **Consequence.** The same lock order everywhere is the defence the PostgreSQL documentation recommends against deadlocks [PG-LOCK]. The user sees a clear message and nothing is half saved.

DR-17. The covers cap is serialised with a row lock

- **Context.** A restaurant may cap how many guests arrive at one time (kitchen pacing). The check is "read covers, then insert", which two requests could pass at once.
- **Choice.** `_lock_period` in `restaurant_service/availability.py` updates the service period row to the value it already has. On Postgres this locks the row, so a second confirm waits until the first commits (under the default Read Committed level, an UPDATE of a row locked by another transaction waits for that transaction to finish [PG-ISO]); on SQLite the update takes the single write lock.
- **Consequence.** The test `test_covers_cap_holds_under_concurrent_confirms` slows the cap read down on purpose and still sees the cap held. Note: the decisions log ("restaurants vertical API", item 6) and `docs/data-model.md` still say the cap could be broken by one party; that text was written before this lock and is now stale (section 14.4).

DR-18. A hand-written SVG seat map

- **Context.** The map is the heart of the product and must work with a mouse, a finger and a keyboard.
- **Options.** (a) A canvas library. (b) A general diagram library. (c) Our own SVG component.
- **Choice.** (c): `frontend/src/seatmap/` (`SeatMap.tsx`, `usePanZoom.ts`, `mapKeyboard.ts`, `describe.ts`). The floor-plan editor reuses the same drawing pieces but is a separate component, because the booking map must never allow dragging (decisions log, "frontend foundation", item 7).
- **Consequence.** Full control over accessibility: the whole map is one tab stop (`role="listbox"`), the arrow keys move a cursor from seat to seat (`aria-activedescendant` points at the current seat, each seat being an `option` with a spoken label), and states use a shape and a mark as well as a colour. The cost is that we wrote pan, zoom and hit testing ourselves; one bug of that kind (a tap that panned the map so the tap missed the table) was found and fixed (decisions log, "fixes after the adversarial review", item 14).

DR-19. Plain CSS with design tokens

- **Context.** Four beginners must be able to explain every style.
- **Choice.** Every colour, space, radius, font size and duration is a CSS custom property in `frontend/src/design/tokens.css`; components live in `frontend/src/design/components/`. A script checks the contrast of every token pair (`npm run contrast`).
- **Consequence.** No CSS framework to learn; one place to change a colour. Today's run: `86 checks, 0 failures.`

DR-20. CSV import as the raw request body

- **Context.** Organisers import guest lists from spreadsheets. FastAPI's form uploads need the `python-multipart` package, which is not on the allowed dependency list.
- **Choice.** The CSV is the raw body (`Content-Type: text/csv`), capped at 200 KB and 1,000 rows, all rows or none (decisions log, "events vertical API", item 1).
- **Consequence.** One dependency fewer. A file with one bad row imports nothing and the answer lists every bad line, so the organiser fixes the file and tries again.

DR-21. The demo reset needs a secret header and nothing else

- **Context.** The demo organiser's password was public in the repository, so any reader could have wiped the graded demo with it.
- **Choice.** `POST /api/demo/reset` accepts only the header `X-Demo-Key` matching `DEMO_KEY`; an empty key rejects everyone. On Postgres the public demo passwords never apply (decisions log, "fixes after the adversarial review (platform)", items 1 and 2).
- **Consequence.** The demo can be reset by the team and by nobody else. The demo links (raw tokens on purpose) live in their own `demo_links` table, not in the audit log.

DR-22. Browser tests with Playwright, added late

- **Context.** For most of the project, browser checks were done by hand and the contract said "no browser test suite" (decisions log, "fixes after the adversarial review (platform)", item 14).
- **Choice.** Today the e2e work added 12 Playwright tests in `frontend/e2e/` (user flows on desktop and phone sizes, and axe accessibility scans), with small Node servers that seed a temporary database and serve the built app with the real `vercel.json` headers.
- **Consequence.** The main flows are now checked in a real browser on every run. The README, the contract (section 2) and the decisions log were updated in the final verification to say so. The new CI job has not yet run on GitHub (not done yet).

5. Architecture and data model

The full architecture, with four diagrams, is in `docs/architecture.md`. The generated entity-relationship diagram is in `docs/data-model.md`, and every endpoint is described in `docs/api.md` and `docs/openapi.json`. This section gives the shape of the system in words.

5.1 The big picture

```
Browser (React app, static files)
   |  same address, /api/... requests
   v
Vercel: one Python function (api/index.py -> FastAPI app)
   |  SQLAlchemy, bound parameters only
   v
Postgres (SQLite on a laptop and in tests)

GitHub Actions cron, every 15 minutes --> POST /api/internal/email/process --> Resend
(or console)
```

There is one URL and one deployment. The browser only ever talks to one address: in development the Vite server forwards `/api` to the backend on port 8000; on Vercel, `vercel.json` rewrites `/api/(.*)` to the Python function and every other path to `index.html` so the React router can handle it.

5.2 Backend packages

Package	Job	Examples
`app/`	App factory, settings, database engine and session, all tables, security helpers	`main.py`, `config.py`, `db.py`, `models.py`, `security.py`
`app/core/`	Shared pieces used by both verticals	`holds.py`, `live.py`, `audit.py`, `privacy.py`, `errors.py`, `layout.py`, `neighbours.py`, `text.py`
`app/routers/`	One file per group of HTTP endpoints, kept thin	`attendee.py`, `events.py`, `restaurants.py`, `bookings.py`, `staff.py`, `emails.py`
`app/events_service/`	Database-side logic for events	`seating.py`, `plans.py`, `recommend.py`, `exact_check.py`, `csv_io.py`
`app/restaurant_service/`	Database-side logic for restaurants	`diner_flow.py`, `holding.py`, `availability.py`, `staff_floor.py`, `whatif.py`, `service_plan.py`, `waitlist.py`
`app/optimiser/`	Pure functions, no database	`events_greedy.py`, `events_anneal.py`, `events_exact.py`, `restaurant_assign.py`, `restaurant_exact.py`
`app/mail/`	Outbox, templates, calendar invites, providers	`outbox.py`, `templates.py`, `ics.py`, `invites.py`, `reminders.py`, `providers.py`

Package	Job	Examples
`app/seed/`	Demo data and the table-creation command	`demo_events.py`, `demo_restaurant.py`, `run.py`, `migrate.py`

Each of these folders has a `READ_ME_FIRST.md` with the key functions, the data flow and likely walkthrough questions (contract section 8).

The rule between layers is simple. A router reads the request (a Pydantic model that rejects unknown fields), checks who is calling, and hands over to a service function. The service function reads rows, builds the optimiser's dataclasses when needed, calls the pure function, writes the result, bumps the live counter and queues any email, all in one transaction. The router turns the result into JSON. Errors are typed (`app/core/errors.py`) and always come back in one shape: `{"error": {"code": "seat_taken", "message": "..."}}`.

5.3 Frontend

The frontend is a React [REACT] single-page app in TypeScript, built with Vite [VITE]. It uses react-router and plain CSS with design tokens.

Folder	Job
`frontend/src/design/`	Tokens and shared components (Button, Field, Dialog, Toast, Badge, Legend, Stepper, States)
`frontend/src/seatmap/`	The SVG map: drawing, pan and zoom, keyboard, spoken descriptions
`frontend/src/api/`	Typed fetch client, conversions from server shapes, the `useLive` hook
`frontend/src/pages/events/`	Attendee steps and the organiser workspace
`frontend/src/pages/restaurant/`	Diner booking steps, manage page, staff floor and tabs
`frontend/src/pages/editor/`	Floor-plan editor with undo and redo

The restaurant client keeps two sets of types: "Server..." types that copy what the backend really sends, and screen types, joined by small `to...` functions in `frontend/src/api/restaurant.ts` (decisions log, "integration", item 5). The test fixtures hold real backend shapes, so the frontend unit tests also check the conversions.

5.4 Data model

The database has 24 tables. The ones that matter most:

- **Shared:** `organisers` (accounts, with role `organiser` or `staff`), `venues`, `zones`, `tables`, `seats`, `seat_neighbours`, `holds`, `live_versions`, `email_outbox`, `audit_log`, `rate_limits`, `demo_links`.
- **Events:** `events`, `guest_groups`, `guests`, `acquaintances` ("knows" pairs), `seat_assignments`, `plan_snapshots` (scenarios and history).
- **Restaurants:** `service_periods`, `turn_times`, `blackout_dates`, `bookings`, `table_slots`, `waitlist_entries`.

Conventions (contract section 4): integer primary keys inside the database; random `public_id` or secret token in URLs; tokens stored as SHA-256 hashes; all timestamps in UTC with a time zone. The generated diagram and the full list of unique rules are in `docs/data-model.md`.

What the demo seeds (counted from a temporary database seeded today with `python -m app.seed.run`, then deleted): two events and one restaurant. "Ana and Bruno's wedding" has 72 guests, 80 seats on 10 tables, and 43 guests already seated; the "Lisbon Data Summit" has 116 guests, 150 seats in 30 rows, and 58 seated; the restaurant "Casa Lumen" has 18 tables and the seed creates 325 bookings for it. All people are invented and all addresses end in `@example.com`.

6. How double booking is made impossible

This is the property the project cares about most: **no seat at one event, and no table in one 15-minute slot, can ever belong to two people.** This section states the claim, gives the argument, and shows the tests that check it, including a test we broke on purpose today.

6.1 The constraints

Table	Rule	Columns	What it forbids
`seat_assignments`	`uq_assignment_seat` (UNIQUE)	`event_id, seat_id`	Two guests on one seat at one event
`seat_assignments`	`uq_assignment_guest` (UNIQUE)	`event_id, guest_id`	One guest owning two seats at one event
`table_slots`	primary key	`table_id, date, slot_index`	Two bookings on one table in the same 15-minute slot
`holds`	`uq_holds_resource` (UNIQUE)	`scope, resource_kind, resource_id, slot_key`	Two live holds on the same seat, or the same table and slot

6.2 The argument

We want to show: whatever order requests arrive in, and however many instances serve them, the stored data never has two owners for one seat or one table slot.

1. **Ownership is a row.** A guest owns seat `s` at event `e` exactly when there is a row `(e, s, guest)` in `seat_assignments`. A booking owns table `t` in slot `k` on day `d` exactly when there is a row `(t, d, k, booking)` in `table_slots`. No other table records ownership; the application never decides "taken" from anything else (contract section 4: no "check then insert" without the constraint behind it).
2. **The database refuses a second row with the same key.** A UNIQUE constraint or primary key means the database will not store two rows that agree on the listed columns. PostgreSQL's documentation states this for unique constraints and primary keys, and adds that rows where a listed column is NULL are treated as different [PG-CONSTR]; SQLite's documentation says the same about NULLs [SQLITE-CT]. So the guarantee holds only if none of the key columns can be NULL. In our tables they cannot: `event_id`, `seat_id`, `table_id`, `date` and `slot_index` are declared as non-optional (`Mapped[int]`, `Mapped[date]`), and `holds.slot_key` is a string that defaults to the empty string. The comment in `models.py` explains why: "SQL treats NULLs as all different, which would let two holds with a NULL slot_key slip past the unique constraint".
3. **The check is made by the database at the moment of writing, for every writer.** Two requests on two different serverless instances share nothing in memory, but they write to the same database, and the database enforces the constraint whichever connection the second row arrives on. If two transactions try to insert the same key at the same time, one commits and the other fails with an integrity error. (On Postgres, the second insert waits for

the first transaction to finish before deciding; this is how unique indexes behave. We did not find a single documentation sentence that states this behaviour for inserts, so we mark it as **from our Postgres test run, not from a quoted source**; the README records the two-guest race on a local Postgres 16.2 giving "200 and a clean 409".)

4. **Every failure becomes a clear answer.** `take_hold` and `commit_or_seat_taken` in `backend/app/core/holds.py` catch `IntegrityError`, roll the transaction back and raise `SeatTaken`, which the API returns as HTTP 409 with code `seat_taken` and a friendly message. Because the whole transaction rolls back, there is never half a booking: no assignment without its email row, no slots without their booking.
5. **Holds cannot defeat the rule.** A hold is only a polite reservation. When a person confirms, `confirm_hold` checks that their own hold is still live and deletes it, and the assignment or slot rows are inserted in the same transaction. If somebody else got there first (for example because the first person's hold had expired and been replaced), the insert breaks the ownership constraint in step 2, so the late confirmer gets 409, not a shared seat.

From 1 to 5: the database holds at most one owner per key, and every path that would add a second owner fails as a whole and tells the user. So double booking is impossible **as long as the database enforces its own constraints**, which is the one thing we rely on.

What this argument does **not** cover:

- The **covers cap** (how many guests may arrive at one time) is not a uniqueness rule; it is a count. It is protected by a row lock instead (DR-17) and tested separately.
- **Whether a seat is suitable** (accessible, reserved tag, blocked) is checked in Python before the insert (`seat_for_guest` in `events_service/seating.py`). A bug there could let the wrong person into a free seat, but it could never put two people in one seat.

6.3 The request flow, step by step

For an event guest (see the sequence diagram in `docs/architecture.md`, section 3):

1. `POST /api/a/{token}/hold {seat_id}` runs `hold_seat`: it takes the event lock (`lock_event`, DR-16), checks the seat suits this guest, then calls `take_hold`, which in one transaction deletes an expired hold on this seat, deletes this guest's own older hold, and inserts the new one. A clash on `uq_holds_resource` means someone else holds it: 409.
2. The page shows a 5-minute countdown (`HoldBar.tsx`).
3. `POST /api/a/{token}/confirm {seat_id}` runs `confirm_seat`: event lock, `confirm_hold` (is my hold still live? then delete it), insert the `seat_assignments` row, queue the confirmation email, keep the guest's reminder in step, write one audit row, then `commit_or_seat_taken`.
4. The commit also carries the live counter bump, so every open map learns about the change only after it is really saved.

For a diner, the same pattern runs per table per slot: `hold_tables` inserts one `holds` row per table per slot (`restaurant_service/holding.py`), and `confirm_booking` checks the covers cap, consumes the holds, inserts the booking and its `table_slots` rows, and commits once (`restaurant_service/diner_flow.py`).

6.4 The tests

Test	What it does	Today
`tests/test_holds.py::test_thirty_threads_race_for_one_hold`	30 threads, each with its own session, start together (a `threading.Barrier`) and try to hold one seat in a file database	passed
`tests/test_holds.py::test_thirty_people_hold_then_confirm_exactly_one_wins`	30 threads hold then confirm the same seat; exactly one ends with the seat, 29 hear `SeatTaken`	passed
`tests/test_attendee_flow.py::test_two_attendees_race_for_one_seat`	Two attendees through the real HTTP API	passed
`tests/test_restaurant_booking_flow.py::test_twenty_threads_book_the_same_table_exactly_one_wins`	20 threads hold and book table T1 at the same time; exactly one booking exists and it owns slots 76 to 81	passed
`tests/test_restaurant_booking_flow.py::test_covers_cap_holds_under_concurrent_confirms`	Cap of 4; three couples confirm at once with the cap read slowed by 0.3 s	passed

The run of `pytest -k "race or threads or concurrent"` today printed `4 passed, 439 deselected, 1 warning in 2.51s` (the second `test_holds.py` test above is not matched by that filter; it passed in the full run).

The race tests use a real SQLite file, not an in-memory database, because an in-memory database is not shared between threads (the test asserts this).

6.5 Can the tests fail? A deliberate break

A test that cannot fail proves nothing. Today we copied the backend to a temporary folder (outside the repository) and edited the copy only:

1. **Removed `uq_holds_resource` from `holds`.** Running the two 30-thread tests gave `1 failed, 1 passed`: `test_thirty_threads_race_for_one_hold` failed at `assert outcomes.count("held") == 1`, because more than one thread now got a hold. The hold-then-confirm test still **passed**: with no hold constraint, the `seat_assignments` constraint still let only one person own the seat. This is the "defence in depth" in step 5 above, observed, not just argued.
2. **Also removed `uq_assignment_seat`.** Now both tests failed: `2 failed, 8 deselected`. With neither constraint, more than one person ended up with the seat.

The copy was deleted afterwards. We did not do the same experiment for the `table_slots` primary key today.

7. The optimisers

There are two optimisers, one per vertical. Both live in `backend/app/optimiser/` as pure functions. Full write-ups, with every formula, are in `docs/optimiser-events.md` and `docs/optimiser-restaurant.md`; this section summarises them and quotes today's measurements.

7.1 Events: the model

Data. Seats `S`, each on a table, with flags blocked, accessible and an optional reserved tag (`vip`, `family`, `speaker`). Guests `G`, each with "needs accessible", an optional tag, an optional group (with or without "keep adjacent") and a set of people they know. `N(s)` is the set of seats next to seat `s`, built from the floor plan when it is saved. `F` is the set of fixed pairs (guest, seat): the choices made by people.

Decision. `x[g, s] = 1` when guest `g` sits on seat `s`. A variable exists only for allowed pairs: seat not blocked, tag empty or equal to the guest's, accessible if the guest needs it.

Hard rules. Each guest at most once; each seat at most once; only allowed pairs; every fixed pair kept exactly.

Score. For each related pair of guests (they know each other in either direction, or share a group), add points depending on where they sit (`Weights` in `events_model.py`):

Term	Points	When
`friend_adjacent`	10	they know each other and sit next to each other
`friend_same_table`	4	they know each other and sit at the same table
`group_adjacent`	15	same group with "keep adjacent", next to each other
`group_same_table`	6	same group, same table
change penalty (re-plan only)	-8 per guest	the guest's seat differs from the previous plan

The terms add up: two friends side by side at one table earn 10 + 4 = 14. `score()` returns the total and one number per term, so the screen can explain a plan ("friends next to each other: 140 points").

How hard is it? The value of one guest's seat depends on where their friends sit, so the score is a sum of products of two decisions. That is the shape of the quadratic assignment problem. The events write-up gives a short reduction of our own: keep only the same-table term, use tables of 3, and a plan reaching the maximum score exists exactly when the friendship graph can be split into triangles, a problem known to be NP-complete. The two citations in that argument (Sahni and Gonzalez 1976; Garey and Johnson 1979) are marked **unverified** in `docs/optimiser-events.md`, and we did not check them today either. The practical consequence is the same: we should not expect an exact method that is fast on every large event, so the app uses a heuristic and keeps an exact solver only as a yardstick on small events.

7.2 Events: greedy, then simulated annealing

Greedy (`events_greedy.py`):

1. Start from the fixed pairs.
2. Order the other guests: reserved tag first, then accessibility needs, then bigger groups. Ties are shuffled by the seeded generator, then a stable sort keeps that order.
3. Put each guest on the free allowed seat with the largest gain (points with guests already seated). A tagged guest takes their own reserved seat when one is free. A guest who does not need an accessible seat only takes one when nothing else is free, so accessible seats stay open for later guests who need them.
4. If a guest still has nowhere to go, try one repair: move a non-fixed guest from an allowed seat to an empty seat they may use, and seat the new guest there. Only if that fails is the guest left unseated.

Simulated annealing (`events_anneal.py`), the method of Kirkpatrick, Gelatt and Vecchi [KIRKPATRICK83]:

- **Move.** Pick a random non-fixed guest and a random seat. If the seat is empty, move there; if another non-fixed guest sits there, swap. Moves that would put anyone on a seat they may not use are skipped, so every plan visited is feasible.
- **Delta.** Only pairs that involve the one or two moved guests can change, so the change in score is computed from those pairs only. A test checks this shortcut against a full rescore on 300 random moves (`test_incremental_delta_matches_full_rescore`).
- **Acceptance.** A move that does not lower the score is always kept. A move that lowers it by `d` is kept with probability `exp(-d / T)`. Early on, when `T` is high, the search may accept worse plans to escape a local optimum; later it becomes strict.
- **Cooling.** `T` starts at 10 (`START_TEMPERATURE`, "about one friend next to friend bonus") and falls geometrically to 0.05 (`END_TEMPERATURE`) over `max_iterations` steps: each step multiplies `T` by `(0.05 / 10) ^ (1 / max_iterations)`.
- **Stopping.** After `max_iterations` (default 20,000). The 15 s limit is a safety cut-off only (DR-11).
- **Result.** The best plan seen, so never worse than the greedy start.

`solve` runs greedy then annealing with one shared `random.Random(seed)`.

Re-plan (`events_replan.py`) keeps every previous seat that is still valid, fills the gaps greedily, then anneals with the change penalty on. `docs/optimiser-events.md` records a test scenario (36 guests, 5 tables of 8, 3 cancellations) where re-plan moved 0 guests and a fresh solve moved 26; the test `test_replan_moves_fewer_guests_than_a_fresh_solve` checks the direction of that result.

Recommend (`events_recommend.py`) ranks every free allowed seat for one guest by gain and builds reasons only from the guest's own list and group (DR-09).

7.3 Events: the exact check

`events_exact.py` has two engines.

- **CP-SAT** from Google OR-tools [ORTOOLS-CPSAT], used when the package imports. Besides `x[g, s]`, each related pair gets two 0/1 variables, "adjacent" and "same table", each allowed to be 1 only when the plan makes it true. The objective is `M * (guests seated) + pair points`, with `M` larger than all pair points together, so seating more guests always comes first. CP-SAT works over integers [ORTOOLS-CPSAT], so points are multiplied by 1,000 and rounded. One worker and a fixed seed make the answer repeatable.
- **Branch and bound** in pure Python, for hosts where OR-tools cannot be installed. It is depth-first over guests, hardest first; the bound adds, for every pair not yet decided, the most that pair could ever earn, so it never cuts off the optimum.

Both report `proven_optimal`. Tests compare both engines with a separate brute force on tiny cases (`test_exact_equals_brute_force`) and with each other (`test_engines_agree`).

In the app, `POST /events/{eid}/optimise/exact-check` runs on one or two tables (at most 16 seats) with a 10 s limit. When the two plans seat a different number of guests, it reports both counts and no percentage gap, because a plan that leaves someone standing could score as many points as one that seats everybody.

7.4 Events: measured gap (real output today)

Command, run from `backend/` on 10 November 2026 (Python 3.12.13, OR-tools 9.15.6755, Apple arm64): `python -B -m scripts.gap_report_events`. The heuristic ran with `max_iterations = 20000`. "Optimal" is CP-SAT; "B&B" is the branch and bound. Real output:

Guests	Tables x seats	Seed	Seated heur/opt	Heuristic score	Optimal score	B&B score	Gap %	Heuristic s	Exact s (engine)	B&B s	Proven
6	2 x 4	0	6/6	28	28	28	0.0	0.02	0.02 (cp-sat)	0.00	yes
6	2 x 4	1	6/6	68	68	68	0.0	0.07	0.01 (cp-sat)	0.00	yes
6	2 x 4	2	6/6	76	76	76	0.0	0.06	0.00 (cp-sat)	0.00	yes
7	2 x 4	0	7/7	142	142	142	0.0	0.06	0.00 (cp-sat)	0.00	yes
7	2 x 4	1	7/7	76	76	76	0.0	0.05	0.00 (cp-sat)	0.00	yes
7	2 x 4	2	7/7	81	81	81	0.0	0.08	0.01 (cp-sat)	0.00	yes
8	2 x 5	0	8/8	92	92	92	0.0	0.07	0.00 (cp-sat)	0.00	yes
8	2 x 5	1	8/8	102	102	102	0.0	0.06	0.01 (cp-sat)	0.00	yes
8	2 x 5	2	8/8	74	74	74	0.0	0.06	0.00 (cp-sat)	0.00	yes
8	3 x 4	0	8/8	88	88	88	0.0	0.06	0.01 (cp-sat)	0.00	yes

Guests	Tables x seats	Seed	Seated heur/opt	Heuristic score	Optimal score	B&B score	Gap %	Heuristic s	Exact s (engine)	B&B s	Proven
8	3 x 4	1	8/8	88	88	88	0.0	0.06	0.01 (cp-sat)	0.00	yes
8	3 x 4	2	8/8	77	77	77	0.0	0.06	0.01 (cp-sat)	0.00	yes
9	2 x 5	0	9/9	109	109	109	0.0	0.07	0.01 (cp-sat)	0.00	yes
9	2 x 5	1	9/9	122	122	122	0.0	0.09	0.01 (cp-sat)	0.01	yes
9	2 x 5	2	9/9	124	124	124	0.0	0.07	0.01 (cp-sat)	0.00	yes
10	3 x 4	0	10/10	105	109	109	3.7	0.09	0.05 (cp-sat)	0.06	yes
10	3 x 4	1	10/10	156	156	156	0.0	0.11	0.08 (cp-sat)	0.66	yes
10	3 x 4	2	10/10	142	142	142	0.0	0.11	0.06 (cp-sat)	0.55	yes
10	2 x 6	0	10/10	117	117	117	0.0	0.09	0.07 (cp-sat)	0.09	yes
10	2 x 6	1	10/10	176	176	176	0.0	0.12	0.09 (cp-sat)	0.64	yes
10	2 x 6	2	10/10	166	166	166	0.0	0.11	0.04 (cp-sat)	0.34	yes

The whole script took 5.01 s of wall time (`time`).

Reading. All 21 instances: the heuristic seated everyone, as the optimum did. In 20 of 21 the heuristic score equals the proven optimum. In one (10 guests, 3 tables of 4, seed 0) it scored 105 against 109, a gap of 4 / 109 = 3.7 %. Both exact engines agreed on every instance.

What this does not show. These are 21 random instances of at most 10 guests. They say little about a 116-guest conference: there, no proven optimum exists to compare with, and the gap is unknown. `docs/optimiser-events.md` reports earlier timing checks (100, 200 and 400 guests) and a 14-guest case where neither exact engine proved optimality in 60 s; we did not re-run those today.

Repeatability. We ran the script a second time and compared the first nine columns (everything except timings) of the two outputs with `cmp`: identical.

7.5 Restaurants: the model

Data. Tables `T`, each with a minimum and maximum party size, a zone, accessible, accepts a high chair, and an optional `combine_group` (tables with the same group may be joined). Parties `P`, each with a size, a half-open interval of 15-minute slots `[a, b)`, needs (accessible, high chairs), an optional zone wish, and optional fixed tables (the diner's own choice).

Candidate choices. For each party, the sets of tables that could host it ignoring time: one table whose size range fits, or 2 or 3 tables of one combine group whose total fits and where every table is needed (otherwise a party of 4 could block three tables for two). Accessibility and high chairs filter the list. A fixed party has exactly one candidate: its own tables. All of this is in one function, `choice_problem`, shared by the heuristic, both exact solvers and the rule checker.

Hard rules. Each party sits at most once; a fixed party sits exactly at its tables; one party per table per slot. In code, the last rule is a dictionary keyed by `(table, slot)` (the `Floor` class), the same idea as the `table_slots` primary key in the database.

Objective (lexicographic). First seat the most covers (guests); then waste the fewest seats (sum of table sizes minus party size, which protects big tables for big parties); then meet the most zone wishes. In Python this is a tuple compared left to right: `(covers, −waste, zones)`. For CP-SAT the tuple becomes one weighted sum whose weights make each level outweigh everything below it (`_objective_weights`).

How hard is it? `docs/optimiser-restaurant.md` states carefully what is known and what is not: special cases are polynomial (bipartite matching; interval scheduling on identical machines), but our problem mixes table eligibility, time intervals and table combinations. The literature claims about that mix are marked **unverified** there; we did not verify them today.

7.6 Restaurants: the algorithms

- `recommend_table` ranks free candidate choices by least waste, then zone wish, then fewer tables, then table ids; it drops combinations when any single table fits; it explains each choice ("Exact fit for 4", "Keeps the table for 6 free for a larger party").
- `assign_service` places fixed parties first, then a seeded greedy (largest parties first, best fit), then a local search with four moves (move, swap, seat a waiting party, push a seated party), keeping a move when the key is not worse. Default budget: 5,000 iterations.
- `whatif_walkin` answers "can we seat a walk-in of N now?" on a copy of the plan: a free table if there is one, otherwise the cheapest set of moves of non-fixed parties, named by party and table labels, never ids.
- `assign_exact` uses CP-SAT, or a branch and bound with a bound that is valid for the lexicographic order.

In the app, `POST /staff/{vid}/optimise/service` moves only bookings the optimiser placed itself (`table_choice` "auto") whose party has not sat down; a diner's own table and seated parties are fixed. The first call is a preview with a `plan_id`; applying needs that `plan_id`, and returns 409 if the floor changed in between.

7.7 Restaurants: measured gap (real output today)

Command, run from `backend/` on 10 November 2026: `python -B -m scripts.gap_report_restaurant` (heuristic `assign_service(seed=seed, max_iterations=5000)`). It printed `OR-tools available: True`. "Covers / waste / zones" is the lexicographic key. Real output:

Small services (CP-SAT and branch and bound)

Tables	Parties	Seed	Covers wanted	Heuristic covers / waste / zones	Optimal covers / waste / zones	Covers gap %	Heuristic optimal	Heuristics	Exact s	Exact agree	Proven
4	6	0	36	14 / 2 / 1	14 / 2 / 1	0.0	yes	0.01	cp-sat 0.00, branch-and-bound 0.00	yes	yes
4	6	1	20	18 / 2 / 1	18 / 2 / 1	0.0	yes	0.02	cp-sat 0.00, branch-and-bound 0.00	yes	yes
4	6	2	29	13 / 1 / 1	13 / 1 / 1	0.0	yes	0.03	cp-sat 0.00, branch-and-bound 0.00	yes	yes
4	6	3	27	11 / 1 / 1	11 / 1 / 1	0.0	yes	0.02	cp-sat 0.00, branch-and-bound 0.00	yes	yes
5	7	0	43	20 / 2 / 2	20 / 2 / 2	0.0	yes	0.03	cp-sat 0.00, branch-and-bound 0.00	yes	yes
5	7	1	25	23 / 3 / 1	23 / 3 / 1	0.0	yes	0.02	cp-sat 0.00, branch-and-bound 0.00	yes	yes
5	7	2	31	18 / 2 / 1	18 / 2 / 1	0.0	yes	0.03	cp-sat 0.00, branch-and-bound 0.00	yes	yes
5	7	3	31	15 / 3 / 1	15 / 3 / 1	0.0	yes	0.02	cp-sat 0.00, branch-and-bound 0.00	yes	yes
6	8	0	50	35 / 3 / 2	35 / 3 / 2	0.0	yes	0.04	cp-sat 0.00, branch-and-bound 0.00	yes	yes
6	8	1	32	30 / 4 / 1	30 / 4 / 1	0.0	yes	0.02	cp-sat 0.00, branch-and-bound 0.00	yes	yes
6	8	2	38	25 / 3 / 2	25 / 3 / 2	0.0	yes	0.03	cp-sat 0.00, branch-and-bound 0.00	yes	yes
6	8	3	39	30 / 4 / 3	30 / 4 / 3	0.0	yes	0.02	cp-sat 0.00, branch-and-bound 0.00	yes	yes
6	10	0	60	37 / 3 / 2	37 / 3 / 2	0.0	yes	0.04	cp-sat 0.00, branch-and-bound 0.00	yes	yes
6	10	1	38	34 / 4 / 2	34 / 4 / 2	0.0	yes	0.03	cp-sat 0.00, branch-and-bound 0.00	yes	yes
6	10	2	51	28 / 6 / 2	28 / 6 / 2	0.0	yes	0.03	cp-sat 0.00, branch-and-bound 0.00	yes	yes

Tables	Parties	Seed	Covers wanted	Heuristic covers / waste / zones	Optimal covers / waste / zones	Covers gap %	Heuristic optimal	Heuristics	Exact s	Exact agree	Proven
6	10	3	48	35 / 3 / 3	35 / 3 / 3	0.0	yes	0.02	cp-sat 0.00, branch-and-bound 0.00	yes	yes
8	12	0	69	41 / 3 / 3	41 / 3 / 3	0.0	yes	0.05	cp-sat 0.01, branch-and-bound 0.01	yes	yes
8	12	1	44	39 / 3 / 3	39 / 3 / 3	0.0	yes	0.03	cp-sat 0.00, branch-and-bound 0.02	yes	yes
8	12	2	59	34 / 12 / 1	34 / 6 / 2	0.0	no	0.05	cp-sat 0.03, branch-and-bound 0.03	yes	yes
8	12	3	61	40 / 2 / 3	40 / 2 / 3	0.0	yes	0.04	cp-sat 0.00, branch-and-bound 0.02	yes	yes

Demo-sized services (CP-SAT only)

Tables	Parties	Seed	Covers wanted	Heuristic covers / waste / zones	Optimal covers / waste / zones	Covers gap %	Heuristic optimal	Heuristic s	Exact s	Exact agree	Proven
12	30	0	159	87 / 13 / 5	87 / 3 / 3	0.0	no	0.17	cp-sat 1.42	yes	yes
12	30	1	138	74 / 6 / 6	74 / 6 / 6	0.0	yes	0.11	cp-sat 0.03	yes	yes
12	30	2	134	70 / 12 / 3	72 / 10 / 3	2.8	no	0.12	cp-sat 0.04	yes	yes
12	30	3	148	74 / 2 / 6	74 / 2 / 6	0.0	yes	0.13	cp-sat 0.04	yes	yes
20	60	0	263	152 / 14 / 11	154 / 12 / 11	1.3	no	0.64	cp-sat 1.49	yes	yes
20	60	1	272	133 / 19 / 10	138 / 26 / 8	3.6	no	0.75	cp-sat 0.97	yes	yes
20	60	2	266	131 / 25 / 6	131 / 17 / 8	0.0	no	0.81	cp-sat 0.99	yes	yes
20	60	3	260	142 / 16 / 11	146 / 12 / 10	2.7	no	0.67	cp-sat 1.03	yes	yes

The whole script took 10.48 s of wall time (`time`).

Reading.

- Small services: the heuristic seated the optimal number of covers in all 20, and matched the full key in 19. In the other one it wasted 12 seats where 6 was possible.
- Demo-sized services: optimal covers in 4 of 8; 1.3 %, 2.7 %, 2.8 % and 3.6 % fewer covers in the other 4. The full key was optimal in 2 of 8.

- CP-SAT proved every demo-sized optimum in under 1.5 s on this machine. That is a measurement on our generator, not a promise.
- In the demo-sized table only one exact engine runs, so "Exact agree" is always yes there and means nothing.

What it means. The restaurant heuristic is good on small services and noticeably weaker on busy ones. Its local search never accepts a worse plan, so it gets stuck in local optima; `docs/optimiser-restaurant.md` records that running 4 to 10 times more iterations changed little. The obvious next step is to reuse simulated annealing (as in the events optimiser) or restarts with other seeds. We kept the simpler rule because it is easy to explain and never returns a plan worse than its greedy start. This is a real weakness and is listed in section 14.

7.8 Why both a heuristic and an exact method

The exact solvers answer "how far from the best could we be?" but cannot be trusted to finish on large inputs. The heuristics always finish fast but cannot say how good they are. Running both on the same small inputs is how we turned "the optimiser is good" into a number. The two exact engines also check each other: they agreed on every instance where both ran today.

8. Real-time design

8.1 The constraint

The planned host runs our backend as a serverless function. Each request may land on a different instance, instances share no memory, and a function may run for at most `maxDuration` seconds (we set 30 in `vercel.json`; Vercel's documentation describes `maxDuration` as the longest time a function can run before it is terminated [VERCEL-DURATION]). A classic real-time design, where the server keeps a list of connected browsers and pushes each change to them, does not fit: the instance that saves a change is usually not the one that holds the other browser's connection.

8.2 The design

The only shared thing is the database, so the design puts the "what changed" signal there:

1. **A counter per scope.** `live_versions` has one row per scope: one per event (`event-<public id>`) and one per restaurant per day (`venue-<public id>-<yyyy-mm-dd>`). Every write adds one to the counter of the scope it touches, in the same transaction as the write (`bump` in `backend/app/core/live.py`). Listeners therefore never see a version for a change that was rolled back.
2. **A short stream.** `GET /api/live/{scope}/stream` is a server-sent event stream [WHATWG-SSE]. It first sends `retry: 1000`, then reads the counter about once a second (`CHECK_EVERY_SECONDS = 1.0`) and sends an event named `version` whenever the number changes. After about 25 seconds (`STREAM_SECONDS = 25.0`) it ends, well inside the 30 s function limit. The browser's `EventSource` reconnects on its own; the HTML standard defines the `retry` field as the reconnection time [WHATWG-SSE].
3. **Polling as a fallback.** `frontend/src/api/useLive.ts` opens the stream first. If it never opens, or errors 3 times (`MAX_ERRORS_BEFORE_FALLBACK = 3`), or is closed, the hook polls `GET /api/live/{scope}/version` every 2 seconds (`POLL_INTERVAL_MS = 2000`).
4. **Only a number travels.** When the version changes, the page fetches its data again through the normal endpoints. So live updates go through exactly the same permission and privacy checks as a page load; the stream itself carries no names.

8.3 Protecting the database

Each open stream reads the database once a second. Two limits keep this bounded (`live.py`): one address may open at most 30 streams a minute (`STREAMS_PER_IP_PER_MINUTE`; one tab reconnects about 2 or 3 times a minute), and one instance serves at most 50 open streams (`MAX_OPEN_STREAMS`). Beyond either, the server answers 429 and the page falls back to polling. The stream is an `async` generator: while it sleeps it holds no worker thread, so open streams cannot starve the normal routes; only the short database read borrows a thread, and each read uses a fresh short session so no connection is held while waiting.

8.4 What is tested and what is not

`tests/test_demo_live.py` covers the version endpoint and the stream on the backend; the frontend unit tests cover the hook's fallback rules. **Not checked:** whether Vercel buffers the stream. If it does, changes would arrive late (at the end of each 25-second stream) instead of falling back to polling. This can only be checked on a real deployment, which is not done yet. No browser test waits for a change made by another user to appear (`docs/test-report.md`, "What is not tested"). The accessibility review also notes that no page shows the "live" or "reconnecting" status that `useLive` returns (`docs/accessibility.md`, heuristic 1).

9. Email

The full design is in `docs/email.md`; the flow diagram is in `docs/architecture.md`, section 5.

9.1 What is sent

Vertical	Emails
Events	Seat confirmed, seat changed, seat released (each with a calendar invite); one reminder per guest per event
Restaurants	Booking confirmed, changed, cancelled (with invite); reminder; waitlist offer

Bodies come from pure functions in `backend/app/mail/templates.py` that return a subject, a plain-text body and an HTML body. Every user value is HTML-escaped in the HTML version. Both versions carry the sender name, a footer saying why the person got the email, and the link printed as text in case the button does not work.

9.2 Outbox, idempotency and retries

- **Same transaction.** `enqueue()` inserts the row without committing, so the email exists only if the booking or seat change is saved (DR-13).
- **One row per email.** The `idempotency_key` column is UNIQUE and the insert uses `ON CONFLICT DO NOTHING`, so queuing the same email twice keeps one row (`test_enqueue_twice_keeps_one_row`, `test_same_key_twice_gives_one_row_and_one_send`).
- **One send per row.** `process_due()` claims a row with one conditional UPDATE that moves it to `sending` and pushes `next_attempt_at` 10 minutes ahead. Only the processor whose UPDATE changed the row sends it (`test_two_processors_at_once_send_each_email_once`). If the processor crashes, the lease runs out and another run takes the row again (`test_crashed_claim_is_taken_again_after_the_lease`). For that rare case the key is also sent to Resend as an `Idempotency-Key` header (`docs/email.md`, section 2, verified there against Resend's documentation).
- **Backoff.** After a failure the row waits 5, 10, 20, then 40 minutes; the fifth failure marks it `failed` (`test_backoff_doubles`, `test_gives_up_after_five_attempts`).
- **Reminders** are rows whose `next_attempt_at` is in the future. For events, `sync_event_reminder()` keeps exactly one reminder per guest per event and rewrites or skips it when the seat changes.
- **No stored links once finished (S9).** A row keeps the reader's raw personal link only while it waits to be sent. When it ends `sent`, `skipped` or `failed`, every personal link in its words, invite and error becomes `[personal link removed]`; each cron run also empties words and invite of finished rows older than 30 days (`test_outbox_retention.py`).

9.3 Calendar invites

`backend/app/mail/ics.py` builds an iCalendar file [RFC5545]. Lines are folded so that none is longer than 75 octets, which RFC 5545 section 3.1 asks for ("Lines of text SHOULD NOT be longer than 75 octets, excluding the line break", checked today). Text values are escaped, times are written in UTC with a `Z`, and each seat or booking has a stable UID so a calendar updates the same entry. A cancellation uses `METHOD:CANCEL` with the same UID and a higher `SEQUENCE`, following iTIP [RFC5546]. The `SEQUENCE` is the number of seconds since 1970 (decisions log, "integration", item 2), so no counter column is needed. How Gmail, Outlook and Apple Calendar treat these files was not checked on real clients (`docs/email.md`, section 4).

9.4 Demo safety and what is not done

Only allowlisted addresses reach a provider (DR-14). The default provider is `console`, which logs one line per email and never the body (the body holds a secret link); since today's security work, the logged address is masked too. The cron call needs the `X-Cron-Secret` header, compared with `hmac.compare_digest`, and is rate limited to 30 calls a minute per address.

Not done yet: no email has been sent through Resend. Resend only sends from a domain the sender owns and has verified (`docs/email.md`, section 5), and the team has not set one up. The scheduled workflow (`.github/workflows/email-cron.yml`) has not run against a deployment, because there is no deployment yet.

The tests for the email system (eight files, `tests/test_mail_*.py` and `tests/test_outbox.py`) passed in today's full run; the e2e work ran them on their own today and recorded `48 passed` (`docs/test-report.md`).

10. Security and privacy

A red-team review of the whole app was done today; its threat model, all 23 findings, the authorisation of each of the 65 routes, and the OWASP and GDPR mappings are in `docs/security.md`. This section gives the design and the outcome.

10.1 Design rules (contract section 7)

Rule	How the code does it
Secrets only from the environment	`backend/app/config.py`. On Postgres the app refuses to start without `SECRET_KEY`, and since today also with one shorter than 32 characters
No string-built SQL	SQLAlchemy queries with bound parameters only
Validated input	Every body is a Pydantic model with `extra="forbid"` and length limits; free text uses `PlainText` (`core/text.py`), which refuses control characters
Passwords	scrypt from the standard library with `n = 2**14`, `r = 8`, `p = 1`, a 16-byte random salt, compared with `hmac.compare_digest` (`security.py`)
Sessions	An HMAC-SHA256 signed value that expires after 12 hours (`SESSION_HOURS = 12`)
Personal links	`secrets.token_urlsafe(32)`, stored only as SHA-256; wrong and revoked tokens give the same 404
Rate limits	Stored in the database so they work across instances (`rate_limits`). For example: login 10 a minute per address, attendee routes 120 a minute, demo reset 5, email processing 30, live streams 30
Browser	CORS allows only `FRONTEND_ORIGIN`; React escapes output and `dangerouslySetInnerHTML` is not used
Headers on every page (`vercel.json`)	`X-Frame-Options: DENY`; a Content-Security-Policy that allows only our own origin and `frame-ancestors 'none'`; `Referrer-Policy: no-referrer` (so `/e/<token>` and `/b/<token>` never leak to other sites); `nosniff`; a Permissions-Policy; and, since today, `Strict-Transport-Security`
Roles	Organisers see their own events only; staff see every venue (single-tenant demo, decisions log, "backend foundation", item 10)

The rate limit is a fixed window per calendar minute, so a burst across a minute boundary can pass up to twice the limit; this is a known and accepted simplification (decisions log, "fixes after the adversarial review (platform)", item 10).

10.2 Outcome of today's review

From `docs/security.md`: no Critical or High findings; 23 findings in total; 9 fixed today (3 Medium, 6 Low), 14 still open (12 Low, 2 for information). Each fix came with a test that failed before the fix (`backend/tests/test_redteam.py`, `test_booking_cap.py`, `test_outbox_retention.py`). The most important ones:

- **Fixed, Medium (S1).** Off Vercel, a caller could send any `X-Forwarded-For` header and dodge every rate limit. Now the header is trusted only when running on Vercel.

- **Fixed, Medium (S8).** One person could fill a restaurant with fake bookings by holding and booking again and again. Now one mailbox may have at most 4 upcoming bookings at a restaurant, and one client address at most 10 booking attempts there per UTC day (an IPv6 address counts by its /64); both answer 429. A second review found that `name+1@` and a fresh IPv6 address in the same /64 slipped through the first version; both are closed. The email is never checked, so the address cap is the real brake, and a classroom on one Wi-Fi shares it: the team should confirm the number 10.
- **Fixed, Medium (S9).** The email outbox kept the raw manage and attendee links for ever. Now a row keeps its link only while waiting to be sent; once sent, skipped or failed, every personal link in words, invite and error becomes `[personal link removed]`, and 30 days later words and invite are emptied. The second review found a link cut in two by invite line folding, and a link quoted in a stored provider error; both are closed.

Dependency audits run today by the security and e2e work: `pip-audit` and `npm audit` both reported no known vulnerabilities (`docs/security.md`, section 7).

10.3 Privacy

- **Data minimisation towards other guests.** Other attendees never receive email, dietary or accessibility data; names pass through `display_name()` (DR-08). The recommendation ranking cannot reveal who listed you (DR-09).
- **Erasure.** An organiser can delete a guest; the row stays as "Deleted guest" with no email, no dietary note, a revoked token and RSVP `no`, so old plan snapshots still resolve, while the seat, holds, acquaintances and the pending reminder's words and address are removed (decisions log, "events vertical API", item 11).
- **Logs.** The audit log never stores tokens or links; the email log masks addresses (`an***@example.com`); since today, IP addresses in `rate_limits` are stored as keyed hashes (S7).
- **Gaps.** Diners cannot ask to be erased, and no data is ever deleted on a schedule (S23), except outbox words and invites, which go 30 days after sending (S9). `docs/security.md`, section 6, maps these to the General Data Protection Regulation [GDPR]. We confirmed today which regulation that is (Regulation (EU) 2016/679 of 27 April 2016) from the EU Publications Office record; the article titles quoted in `docs/security.md` were **not re-checked** today, because the EUR-Lex text did not load from this machine.

The OWASP mapping in `docs/security.md` uses the 2021 edition of the OWASP Top 10. We confirmed today that its ten categories are A01 Broken Access Control to A10 Server-Side Request Forgery [OWASP-2021], and that OWASP has since published a 2025 edition [OWASP-TOP10]; the mapping has not been redone against the 2025 list.

11. Testing, with real results

`docs/test-report.md` describes every layer of tests, how to run it and what CI runs. This section gives today's numbers and what they mean.

11.1 Layers

Layer	Tool	Where	What it proves
Unit	pytest; vitest with jsdom	`backend/tests/`, `frontend/src/**/*.test.ts(x)`	Each module does its job: scores, rules, privacy, tokens, components, conversions
Property-based	Hypothesis [HYPOTHESIS]	12 tests in 5 files	Rules hold for many random inputs: no two guests on one seat, accessible needs met, exact equals brute force on tiny cases
Concurrency	pytest with threads and a file database	5 tests (section 6.4)	Exactly one winner when many people grab the same thing
API end to end	FastAPI `TestClient`	`tests/test_e2e_slice.py`	Both verticals walk through the real HTTP API from a demo reset
Browser end to end	Playwright [PLAYWRIGHT] with axe	`frontend/e2e/` (12 tests)	Real flows in Chromium at desktop and phone sizes; no page error or CSP violation; no axe finding at WCAG 2.1 A or AA on 6 pages
Red team	pytest	`tests/test_redteam.py` (12 tests), `tests/test_booking_cap.py` (13), `tests/test_outbox_retention.py` (13)	Each fixed security finding stays fixed
Static	ruff, TypeScript, contrast script, audits		Style, types, colour contrast, known vulnerable packages

11.2 Results of the final verification run (10 November 2026)

Run on macOS, Python 3.12.13, Node 24.17.0, after the last code change of the day (the S8 and S9 fixes and their second review). Earlier runs for this report gave 443 backend tests; the S8 and S9 work added 26. The frontend count was 132 before the final verification added one unit test (dates in British English, section 12.4) and 133 before the S8 work added one (the booking form shows a 429's words).

Command	Real output
`cd backend && ../.venv/bin/python -m pytest -p no:cacheprovider`	`469 passed, 1 warning in 31.52s`
the same with `SEATPLAN_TEST_POSTGRES` (throwaway Postgres 16.2 cluster, deleted afterwards)	`468 passed, 1 skipped, 1 warning in 62.08s (0:01:02)`

Command	Real output
`... -k "race or threads or concurrent"`	`4 passed, 465 deselected, 1 warning in 2.14s`
`... -m hypothesis`	`12 passed, 457 deselected, 1 warning in 1.76s`
`... tests/test_redteam.py`	`12 passed, 1 warning in 1.29s`
`... tests/test_booking_cap.py`; `... tests/test_outbox_retention.py`	`13 passed, 1 warning in 2.65s`; `13 passed, 1 warning in 1.07s`
`cd backend && ../.venv/bin/ruff check .`	`All checks passed!`
`cd backend && ../.venv/bin/ruff format --check .`	`138 files already formatted`
`cd frontend && npm test`	`Test Files 15 passed (15)`, `Tests 134 passed (134)`
`cd frontend && npm run typecheck`	exit code 0
`cd frontend && npm run build` (output to a temporary folder)	`✓ built in 133ms`
`cd frontend && npm run contrast`	`86 checks, 0 failures.`
`cd frontend && npm run e2e`, 3 runs	`12 passed (10.8s)`, `12 passed (10.4s)`, `12 passed (9.5s)`
`npm audit --audit-level=high`; `pip-audit -r requirements.txt --no-deps --disable-pip`	`found 0 vulnerabilities`; `No known vulnerabilities found`
`python -B -m scripts.gap_report_events`	section 7.4
`python -B -m scripts.gap_report_restaurant`	section 7.7
Mutation check on a temporary copy (section 6.5)	`1 failed, 1 passed` then `2 failed`

The one pytest warning comes from Starlette's test client asking for an extra package; it does not come from SeatPlan code.

11.3 Results run today by the other work streams

These are quoted from the documents that recorded them today.

Check	Output	Source
Browser tests by the e2e work, before the final accessibility fixes, 3 runs	`12 passed (12.3s)`, `12 passed (10.2s)`, `12 passed (9.8s)`	e2e work report
A browser test made to fail on purpose (expected 200 instead of 409)	`Expected: 200 / Received: 409`	`docs/test-report.md`
Red-team tests before the fixes	`8 failed, 1 passed`	`docs/security.md`
Red-team tests after the fixes	`12 passed`	`docs/security.md`

The backend count went from 431 to 443 today because the security work added 12 red-team tests, then to 469 with the S8 and S9 tests (7 and 9 from the fixes, 6 and 4 from the second review). Earlier in the day the full suite ran once on a local Postgres 16.2 (`430 passed, 1 skipped, 1 warning`, 431 tests); the final verification ran it again on a throwaway cluster (section 11.2).

11.4 What the tests do not cover

From `docs/test-report.md`: a real Vercel deployment; Postgres in CI; real email delivery; live updates arriving in a browser from another user; a hold expiring in a browser; the floor-plan editor and several staff and organiser tabs in a browser; browsers other than Chromium; real phones; screen readers; usability sessions. The new `e2e` job and the gitleaks secret scan have not yet run on GitHub.

12. User experience and accessibility

The UX reasoning per vertical is in `docs/ux-events.md` and `docs/ux-restaurant.md`; the design system is in `docs/design-system.md`; the accessibility statement, with every WCAG 2.2 A and AA criterion, is in `docs/accessibility.md`. User guides for each role are in `docs/user-guide-*.md`.

12.1 Main UX choices

- **One decision per screen** for attendees and diners. The attendee flow is a stepper: welcome and RSVP, people you know (can be skipped), seat, done. The diner flow is party and date, time, table, details.
- **Reasons everywhere.** Every recommended seat or table says why, in venue words ("Next to 2 people you know", "Exact fit for 4", "Keeps the table for 6 free"). The optimiser's "score" appears only on the organiser's Optimise tab.
- **Holds you can see.** A countdown bar shows how long the seat or table is held (`HoldBar.tsx`, `HoldCountdown.tsx`).
- **Errors that say what to do next.** A 409 for a taken seat carries the next best options; for restaurants the error object includes alternative times and tables (decisions log, "restaurants vertical API", item 5).
- **Previews before changes.** Auto-fill, re-plan and the staff service plan show a preview ("Preview, not applied") before anything is written; what-if never writes.
- **Undo.** The organiser can undo and redo plan changes; the floor-plan editor has its own undo history.

12.2 Accessibility built in

- The seat map is keyboard-operable: one tab stop, arrow keys between seats, zoom and fit buttons with labels (DR-18).
- Seat states are shown by shape and mark, not by colour alone; a legend sits next to every map.
- Touch targets use a 44 px token (`--touch-target: 44px` in `frontend/src/design/tokens.css`), above the 24 by 24 CSS pixel minimum of WCAG 2.2 success criterion 2.5.8 [WCAG22-258].
- Reduced motion is respected in `tokens.css` and `base.css` (`prefers-reduced-motion`).
- Contrast of every token pair is checked by a script: `86 checks, 0 failures.` (`docs/accessibility.md`).
- The e2e work's axe scans found no WCAG 2.1 A or AA problem on the 6 pages scanned (`docs/test-report.md`).

12.3 Heuristic review

`docs/accessibility.md`, section 4, reviews the app against Nielsen's ten usability heuristics [NIELSEN]; the review was done by reading code and UX notes, not with users. Its main gaps: no "live" or "reconnecting" sign (heuristic 1); "score" and "gap %" are optimiser words (heuristic 2); no undo after a staff action (heuristic 3); the staff walk-in and what-if lists stop at 12 people (heuristic 5); no help page in the app (heuristic 10).

12.4 Known accessibility risks

`docs/accessibility.md` makes no conformance claim against WCAG 2.2 [WCAG22] and listed five proposed fixes. The first three were done in the final verification on 2026-11-10, and the unit and browser tests passed afterwards:

1. Sticky header and hold bars could hide a focused control (risk under 2.4.11 Focus Not Obscured, which asks that a focused component is "not entirely hidden due to author-created content" [WCAG22]). Done: `design/base.css` sets `scroll-padding-top` and `scroll-padding-bottom` on `html`. Not checked with the keyboard in a real browser.
2. The recommended-seat halo animation ran without end unless "reduce motion" was on (risk under 2.2.2). Done: it now breathes twice (3.2 s) and stops.
3. Dates were formatted with the browser's locale inside an English page (3.1.2). Done: event dates use `"en-GB"`, with a vitest test.
4. Still open: no page shows the live-update status.
5. Still not checked: screen readers, 200 % zoom, 320 px reflow, text spacing and real touch devices.

12.5 Usability evidence

Not done yet. No usability session with real people has taken place. A full kit is ready in `docs/usability/`: how to run a session, a script of 19 tasks across the four roles, a consent form, an observer sheet, the standard System Usability Scale questionnaire, and a results template whose tables are empty and marked "TO BE FILLED AFTER REAL SESSIONS". This report therefore makes no claim about how easy SeatPlan is to use.

Usability session	Date	Who (role)	Task	What went wrong	Change made

13. Deployment

The deployment guide is `docs/deployment.md`; the operations runbook is `docs/runbook.md`. **Every step there is "not done yet": SeatPlan has never been deployed.** This section describes the plan and what was checked without a deployment.

13.1 The plan

1. Create a Postgres database for the Vercel project and set the environment variables: `DATABASE_URL`, `SECRET_KEY` (at least 32 characters), `FRONTEND_ORIGIN`, `PUBLIC_BASE_URL`, and the demo and email ones needed (README, "Deploying to Vercel").
2. Create the tables once from a laptop with `python -m app.seed.migrate` (DR-06).
3. Optionally seed the demo with `python -m app.seed.run` and private demo passwords.
4. Push to `main`. Vercel builds the frontend (`cd frontend && npm ci && npm run build`, output `frontend/dist`) and serves `api/index.py` as one Python function with `maxDuration: 30`.
5. Add two GitHub secrets (`SEATPLAN_URL`, `CRON_SECRET`) so the email cron can call the deployed site.

13.2 What was checked against the documentation today

- Vercel's Python runtime runs ASGI applications such as FastAPI as Vercel Functions; an entry file defines a top-level `app`; dependencies come from `requirements.txt` or `pyproject.toml`; Python 3.12 is the default version; `excludeFiles` under `functions` in `vercel.json` keeps tests and other files out of the bundle [VERCEL-PY]. `api/index.py`, `requirements.txt` and `vercel.json` follow this.
- For Python, `maxDuration` is set in the `functions` object of `vercel.json` [VERCEL-DURATION]. Note: that page, read today, lists default and maximum durations of 300 s and more on current plans with fluid compute. Our 30 s setting is a choice we made, not a platform default, and the 15 s optimiser cut-off was sized against it (DR-11).
- Vercel's Python page states that Python functions support streaming responses [VERCEL-PY]. Whether the live stream is delivered unbuffered on a real deployment is still **not checked**.

13.3 Continuous integration

`.github/workflows/ci.yml` runs four jobs on every pull request and every push to `main`: `backend` (ruff, format check, pytest, pip-audit), `frontend` (install, typecheck, vitest, build, contrast, npm audit), `e2e` (new today) and `secrets` (gitleaks over the git history). The `e2e` job and the gitleaks scan have not yet been seen running on GitHub.

14. Limits and what is not done

This section lists, in one place, everything we know is missing or weak. The risk log (`docs/risk-log.md`) gives likelihood, impact and an owner role for 19 risks; the revision plan (`docs/team/revision-plan.md`) orders the fixes.

14.1 Not done yet

Item	Status	Where it is prepared
Deployment on Vercel with Postgres	Not done yet	`docs/deployment.md`, `vercel.json`, `api/index.py`
Real email through Resend	Not done yet; needs a verified sending domain	`docs/email.md`
Usability sessions with real people	Not done yet	`docs/usability/`
Screen-reader, real touch-device, 200 % zoom and 320 px checks	Not done yet	`docs/accessibility.md`
The `e2e` and gitleaks CI jobs seen running on GitHub	Not done yet	`.github/workflows/ci.yml`
Rehearsed and timed demo; presentation date	Not done yet	`docs/team/demo-script.md`
Each member showing they can explain their part	Not done yet	`docs/team/tasks.md`, `docs/team/walkthrough-qa.md`, `docs/team/ai-use.md`

14.2 Technical limits

1. **Live updates on Vercel are unchecked.** If the platform buffers the stream, changes arrive up to 25 s late instead of falling back to polling (section 8.4).
2. **The restaurant heuristic is weak on busy services.** It reached the optimal covers in only 4 of 8 demo-sized services today, up to 3.6 % below (section 7.7). Simulated annealing or restarts would be the next step.
3. **The events gap is only known for small events** (at most 10 guests in today's run). For a large conference, the distance from the optimum is unknown.
4. **No migration tool.** A new column on an existing Postgres table needs a hand-written change (DR-06).
5. **Single tenant.** Staff see every venue; any organiser can read the whole email log (security findings S11, S12, accepted for the demo).
6. **A restaurant stay cannot cross midnight** (decisions log, "restaurants vertical API", item 9).
7. **Combined tables** are at most 3 from one combine group; physical distance is not checked; `min_party` is checked only for single tables (`docs/optimiser-restaurant.md`, section 5).

8. **Holds are not swept in bulk.** An expired hold row stays until someone takes the same resource. This does not affect correctness (it is deleted in that transaction) but leaves rows behind.
9. **The rate limit window is fixed per calendar minute,** so a burst across a minute boundary can pass up to twice the limit.
10. **Open security findings:** twelve Low and two for information, none Medium (`docs/security.md`). The S8 booking caps are soft limits that an attacker with many addresses can still go around, and they also hit honest diners sharing one Wi-Fi address after 10 bookings a day.
11. **Time limits were sized for a 30 s function.** If the project setting on Vercel differs, the 15 s optimiser cut-off and the exact-check limits still fit, but they have not been tried there.

14.3 Limits of the evidence

- The race tests run on SQLite in CI. The Postgres run of the whole suite was done once, before today, on a laptop.
- The NP-hardness citations and the restaurant complexity claims in the optimiser write-ups are marked unverified there and were not checked for this report.
- The personas are assumptions, not research.
- Browser tests run in Chromium only.

14.4 Documents that were out of date

Found while writing this report. All were corrected in the final verification on 2026-11-10:

- `docs/decisions-log.md`, "restaurants vertical API", item 6, and `docs/data-model.md` said the covers cap could be broken by one party. `_lock_period` prevents this and `test_covers_cap_holds_under_concurrent_confirms` checks it (DR-17). Both now say so.
- `README.md` and `docs/CONTRACT.md` (section 2) said there was no browser test suite (DR-22), and the README test report showed older numbers. Both now name the Playwright suite and today's results.
- `docs/decisions-log.md`, "restaurant screens", item 13, said a `describe` override on `SeatMap` was still requested; it exists, and the log now says so.

15. Lessons

1. **Put the rule where it cannot be skipped.** Our first instinct was to check "is this seat free?" in Python. Moving the rule into the database made it hold on every path, including paths we had not written yet, and made the proof short (section 6). The deliberate break today showed the value of having two constraints: removing one still left the seat safe.
2. **A test is only evidence if it can fail.** Several of our strongest tests were checked by breaking the code on purpose: the race tests today, the email double-send test (`docs/email.md`), the browser test (`docs/test-report.md`) and every red-team test (`docs/security.md`).
3. **Measure the optimiser instead of describing it.** Before the gap report, we would have said the restaurant heuristic was "near optimal". The numbers say it is optimal on small services and up to 3.6 % short on busy ones. The report also caught an early version whose move set could not escape the greedy start (`docs/optimiser-restaurant.md`, section 3).
4. **Serverless changes the design, not just the hosting.** Holds, live updates, emails, rate limits and table creation all had to work without shared memory and with many instances starting at once. Each one ended up in the database (DR-02, DR-06, DR-12, DR-13).
5. **Repeatability needs a decision.** Stopping on an iteration budget instead of a clock is a small choice that made every optimiser test stable and the gap report reproducible (DR-11).
6. **Privacy leaks hide in rankings.** Hiding a number is not enough when the order itself reveals it (DR-09). We only found this in review.
7. **Write down every deviation.** The decisions log let four people build nine modules in parallel and still merge them; it also shows where it went stale (section 14.4), which tells us to update it in the same change as the code.
8. **Say "not done yet" out loud.** The deployment, real email and usability sessions are not done. Writing that plainly is more useful to the reader than a vague "deployed and tested".

16. Traceability matrix

Status words: **done** (in the code and covered by at least one passing test today), **partial** (in the code, but something is missing or unchecked), **not done**. Test names are in `backend/tests/` unless they start with `frontend/`. "Docs" points to where the design is explained.

ID	Requirement	Code	Tests	Docs	Status
R1	Choose own seat or table on a live 2D map	`frontend/src/seatmap/SeatMap.tsx`; `pages/events/SeatStep.tsx`; `pages/restaurant/StepTable.tsx`	`frontend/src/seatmap/SeatMap.test.tsx`; `frontend/e2e/attendee.spec.ts`, `restaurant.spec.ts`, `mobile.spec.ts`	`docs/ux-events.md`, `docs/ux-restaurant.md`	done (no real-device or screen-reader test)
R2	Two verticals share one core	`backend/app/core/`, `models.py`, one `SeatMap`	`test_holds.py`, `test_models.py`	Section 5; `docs/architecture.md`	done
R3	Personal link and RSVP	`routers/attendee.py`, `events_service/seating.py`	`test_attendee_flow.py::test_full_attendee_flow`, `::test_rsvp_no_releases_the_seat`	`docs/product-spec.md` A1, A2	done
R4	"Who I know" and suggestions with reasons	`events_service/recommend.py`, `optimiser/events_recommend.py`	`test_attendee_flow.py::test_recommendations_name_only_people_i_know`, `test_optimiser_events_search.py::test_recommend_ranks_by_gain_and_explains`	DR-09; `docs/optimiser-events.md`	done
R5	See who sits where within privacy rules	`core/privacy.py`	`test_privacy.py`, `test_attendee_privacy.py::test_names_follow_the_privacy_mode`	DR-08	done
R6	Organiser sees full map and who has not chosen	`routers/events.py`, `pages/events/OrganiserPage.tsx`	`test_events_organiser.py::test_event_detail_shows_real_names_and_who_has_not_chosen`; `frontend/e2e/organiser.spec.ts`	`docs/user-guide-organiser.md`	done
R7	Add guests, CSV import and export	`events_service/csv_io.py`, `guests.py`	`test_events_organiser.py::test_csv_import_reports_every_bad_row_and_imports_nothing`, `::test_csv_import_then_export`; `frontend/src/pages/events/__tests__/csv.test.ts`	DR-20	done

ID	Requirement	Code	Tests	Docs	Status
R8	Auto-fill	`optimiser/events_greedy.py`, `events_anneal.py`; `events_service/plans.py`	`test_optimiser_events_search.py`, `test_events_plans.py`	Section 7.2	done
R9	Re-plan with few moves	`optimiser/events_replan.py`	`test_optimiser_events_search.py::test_replan_moves_fewer_guests_than_a_fresh_solve`	Section 7.2	done
R10	Scenarios, compare, apply, undo, redo, what-if	`events_service/plans.py`	`test_events_plans.py::test_scenarios_list_and_compare`, `::test_undo_and_redo_never_touch_guest_choices`, `::test_whatif_scores_moves_without_saving`	DR-15	done
R11	Exact check and measured gap	`optimiser/events_exact.py`, `restaurant_exact.py`; `events_service/exact_check.py`; `restaurant_service/service_plan.py`; `backend/scripts/`	`test_optimiser_events_exact.py`, `test_optimiser_restaurant_exact.py`, `test_staff_optimise.py`	Sections 7.4, 7.7	done (gap known only on small and demo-sized inputs)
R12	Diner picks date, party, time, table or "choose for me"	`restaurant_service/diner_flow.py`, `availability.py`, `map_view.py`; `pages/restaurant/BookingPage.tsx`	`test_restaurant_slots.py`, `test_restaurant_booking_flow.py`; `frontend/src/pages/restaurant/__tests__/booking.test.tsx`; `frontend/e2e/restaurant.spec.ts`	`docs/user-guide-diner.md`	done (no stay across midnight)
R13	Manage link and waitlist	`routers/bookings.py`, `restaurant_service/manage_flow.py`, `waitlist.py`	`test_bookings_manage.py::test_cancel_frees_the_table_and_emails`, `::test_waitlist_position_and_estimated_wait`	`docs/product-spec.md` D5	done
R14	Staff floor, status, move and combine	`restaurant_service/staff_floor.py`; `pages/restaurant/StaffFloor.tsx`	`test_staff_endpoints.py::test_floor_follows_the_booking_status`, `::test_move_checks_the_rules_and_combines_tables`	`docs/user-guide-staff.md`	done
R15	Walk-ins and what-if	`restaurant_service/whatif.py`, `optimiser/restaurant_assign.py`	`test_staff_endpoints.py::test_walkin_is_placed_by_the_optimiser`, `::test_whatif_shows_who_would_have_to_move`	Section 7.6	done
R16	Dashboard and schedule	`restaurant_service/dashboard.py`, `schedule_edit.py`	`test_staff_endpoints.py::test_dashboard_numbers_on_a_hand_countable_day`, `::test_schedule_read_and_update`	`docs/user-guide-staff.md`	done

ID	Requirement	Code	Tests	Docs	Status
R17	Staff service optimiser, preview then apply	`restaurant_service/service_plan.py`	`test_staff_optimise.py` (7 tests)	Section 7.6	partial (heuristic below optimum on busy services)
R18	Floor-plan editor	`frontend/src/pages/editor/`; `core/layout.py`, `neighbours.py`	`test_venues.py::test_layout_save_builds_neighbours`; `frontend/src/pages/editor/editor.test.tsx`	`docs/product-spec.md` 3.1	partial (no browser test of the editor)
R19	Emails with calendar invites and reminders	`backend/app/mail/`	`test_mail_*.py`, `test_outbox.py`; `test_mail_ics.py::test_long_lines_fold_at_75_octets_without_splitting_characters`	Section 9; `docs/email.md`	partial (no real email sent)
H1	No double booking	`models.py` constraints; `core/holds.py`	Section 6.4 (5 tests) and the deliberate break in 6.5	Section 6; `docs/data-model.md`	done
H2	Five-minute holds	`core/holds.py` (`HOLD_MINUTES = 5`)	`test_holds.py::test_hold_lasts_five_minutes`, `::test_confirm_after_expiry_fails`	DR-02	done (expiry not tested in a browser)
H3	Human choices and hard rules never broken	`events_service/problem.py`, `plans.py`; `optimiser/restaurant_model.py`	`test_optimiser_events_rules.py`, `test_optimiser_events_search.py::test_anneal_never_moves_fixed_guests`, `test_optimiser_restaurant_assign.py::test_fixed_party_never_moves_even_when_moving_would_score_better`, Hypothesis property tests	DR-15; section 7	done
H4	Repeatable results	`random.Random(seed)`; iteration budgets	`test_optimiser_events_search.py::test_solve_same_seed_same_result`, `test_optimiser_restaurant_assign.py::test_same_seed_gives_the_same_plan`	DR-11; Appendix A	done
H5	No private data to other attendees	`core/privacy.py`, `routers/attendee.py`	`test_attendee_privacy.py::test_people_entries_carry_only_safe_fields`, `test_attendee_flow.py::test_recommendations_do_not_reveal_who_listed_me`	Section 10.3	done
N1	Runs on Vercel, one project	`vercel.json`, `api/index.py`	`test_optimiser_events_search.py::test_safety_cut_off_fires_before_the_platform_limit` (reads `vercel.json`)	Section 13; `docs/deployment.md`	not done (never deployed)

ID	Requirement	Code	Tests	Docs	Status
N2	Live updates on serverless	`core/live.py`, `frontend/src/api/useLive.ts`	`test_demo_live.py::test_stream_sends_version_and_ends`; `frontend/src/api/useLive.test.tsx`	Section 8	partial (unchecked on Vercel; no status shown)
N3	Security basics	`security.py`, `config.py`, `vercel.json`, `restaurant_service/booking_caps.py`, `mail/outbox.py`	`test_security.py`, `test_redteam.py`, `test_booking_cap.py`, `test_outbox_retention.py`	Section 10; `docs/security.md`	partial (Low findings open)
N4	Accessibility	`seatmap/mapKeyboard.ts`, `design/tokens.css`	`frontend/e2e/accessibility.spec.ts`; `npm run contrast`	Section 12; `docs/accessibility.md`	partial (no assistive-technology test)
N5	Standard tests and end to end	`backend/tests/`, `frontend/src/**/*.test.*`, `frontend/e2e/`	Section 11	`docs/test-report.md`	done (CI e2e job not yet seen running)
N6	Explainable code and docs	`READ_ME_FIRST.md` in each package; `docs/`	`ruff check`	`docs/team/walkthrough-qa.md`	partial (walkthroughs not yet held)
N7	Fake people only	`backend/app/seed/`	`test_events_seed.py`, `test_restaurant_seed.py`	Contract 7	done
N8	Few dependencies	`requirements.txt`	`pip-audit`	Contract 2	done
U1	Usability evidence	none yet	none	`docs/usability/`	not done

16.1 The brief's own criteria (to be filled in by the team)

The official brief is not in the repository. Paste each criterion's exact words below and point to the rows above.

Criterion from the brief (exact words)	Rows above	Status

17. References

Every source below was fetched on 10 November 2026 and checked to exist and to say what we use it for. Sources we could not fetch are marked.

[FASTAPI] FastAPI documentation, home page. https://fastapi.tiangolo.com/ (accessed 10 November 2026).

[SQLA-CONSTR] SQLAlchemy 2.0 Documentation, "Defining Constraints and Indexes" (`UniqueConstraint`: "A table-level UNIQUE constraint"). https://docs.sqlalchemy.org/en/20/core/constraints.html (accessed 10 November 2026).

[SQLA-TX] SQLAlchemy 2.0 Documentation, "Transactions and Connection Management". https://docs.sqlalchemy.org/en/20/orm/session_transaction.html (accessed 10 November 2026).

[PG-CONSTR] PostgreSQL 18 Documentation, section 5.5 "Constraints" (unique constraints; "two null values are not considered equal in this comparison"). https://www.postgresql.org/docs/current/ddl-constraints.html (accessed 10 November 2026).

[PG-ISO] PostgreSQL 18 Documentation, section 13.2 "Transaction Isolation" (Read Committed is the default; an UPDATE waits for a concurrent transaction that has updated or locked the row; serialization failures return SQLSTATE 40001). https://www.postgresql.org/docs/current/transaction-iso.html (accessed 10 November 2026).

[PG-LOCK] PostgreSQL 18 Documentation, section 13.3 "Explicit Locking" (deadlocks are detected and resolved by aborting one transaction; acquire locks in a consistent order; advisory locks). https://www.postgresql.org/docs/current/explicit-locking.html (accessed 10 November 2026).

[SQLITE-CT] SQLite, "CREATE TABLE" ("For the purposes of UNIQUE constraints, NULL values are considered distinct from all other values, including other NULLs"). https://www.sqlite.org/lang_createtable.html (accessed 10 November 2026).

[REACT] React documentation, "Quick Start". https://react.dev/learn (accessed 10 November 2026). The root page https://react.dev/ answered HTTP 404 to our fetch tool, so we cite the Quick Start page.

[VITE] Vite, "Getting Started". https://vite.dev/guide/ (accessed 10 November 2026).

[VERCEL-PY] Vercel, "Using the Python Runtime with Vercel Functions" (ASGI apps such as FastAPI; top-level `app`; dependencies; Python 3.12 default; streaming; `excludeFiles`). https://vercel.com/docs/functions/runtimes/python (accessed 10 November 2026; page states last updated 2026-08-12).

[VERCEL-DURATION] Vercel, "Configuring Maximum Duration for Vercel Functions" (`maxDuration` in the `functions` object of `vercel.json` for Python). https://vercel.com/docs/functions/configuring-functions/duration (accessed 10 November 2026; page states last updated 2026-08-24).

[ORTOOLS-CPSAT] Google OR-Tools, "CP-SAT Solver" (CP-SAT works over the integers). https://developers.google.com/optimization/cp/cp_solver (accessed 10 November 2026). The OR-tools version used today was 9.15.6755 (printed by `ortools.__version__`).

[WCAG22] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2", W3C Recommendation, 12 December 2024 (success criterion 2.4.11 Focus Not Obscured (Minimum)). https://www.w3.org/TR/WCAG22/ (accessed 10 November 2026).

[WCAG22-258] W3C, "Understanding Success Criterion 2.5.8: Target Size (Minimum)" ("at least 24 by 24 CSS pixels", with five exceptions). https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html (accessed 10 November 2026).

[NIELSEN] Nielsen, J. "10 Usability Heuristics for User Interface Design". Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/ (accessed 10 November 2026).

[OWASP-2021] OWASP, "OWASP Top 10:2021", introduction listing A01:2021 Broken Access Control to A10:2021 Server-Side Request Forgery. https://top10.owasp.org/2021/A00_2021_Introduction (redirected from https://owasp.org/Top10/2021/; accessed 10 November 2026).

[OWASP-TOP10] OWASP, "OWASP Top 10" (now redirects to the 2025 edition). https://top10.owasp.org/ (accessed 10 November 2026). The list of 2025 categories was not read, because the page we fetched was only a redirect notice.

[GDPR] Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation). Record checked at the EU Publications Office: https://op.europa.eu/en/publication-detail/-/publication/3e485e15-11bd-11e6-ba9a-01aa75ed71a1 (accessed 10 November 2026). The official text on EUR-Lex (https://eur-lex.europa.eu/eli/reg/2016/679/oj) returned an automated challenge page and an empty body to our tools: **text not verified today**, so no article wording is quoted in this report.

[RFC5545] Desruisseaux, B. (Ed.), "Internet Calendaring and Scheduling Core Object Specification (iCalendar)", RFC 5545, September 2009. https://www.rfc-editor.org/rfc/rfc5545 (accessed 10 November 2026).

[RFC5546] Daboo, C. (Ed.), "iCalendar Transport-Independent Interoperability Protocol (iTIP)", RFC 5546, December 2009. https://www.rfc-editor.org/rfc/rfc5546 (accessed 10 November 2026).

[KIRKPATRICK83] Kirkpatrick, S., Gelatt, C. D. Jr. and Vecchi, M. P. (1983). "Optimization by Simulated Annealing". Science, 220(4598), 671 to 680. DOI 10.1126/science.220.4598.671. Metadata checked through the Crossref API, https://api.crossref.org/works/10.1126/science.220.4598.671 (accessed 10 November 2026). The publisher page (https://www.science.org/doi/10.1126/science.220.4598.671) answered HTTP 403 to our fetch, so the paper's text was not read today; we cite it only for the method's name and origin.

[WHATWG-SSE] WHATWG, HTML Living Standard, section 9.2 "Server-sent events" (the `retry` field sets the reconnection time). https://html.spec.whatwg.org/multipage/server-sent-events.html (accessed 10 November 2026).

[HYPOTHESIS] Hypothesis documentation, "Welcome to Hypothesis!". https://hypothesis.readthedocs.io/en/latest/ (accessed 10 November 2026).

[PLAYWRIGHT] Playwright, "Installation". https://playwright.dev/docs/intro (accessed 10 November 2026).

Not cited, because not checked today: the complexity results named in the optimiser write-ups (Sahni and Gonzalez 1976; Garey and Johnson 1979; Arkin and Silverberg 1987; Kolen, Lenstra, Papadimitriou and Spieksma 2007) are marked unverified in `docs/optimiser-events.md` and `docs/optimiser-restaurant.md`, and this report does not rely on them. The Resend documentation is cited, with its own check dates, in `docs/email.md`.

Appendix A. Commands run for this report (10 November 2026)

All run on the team's Mac. Temporary files were written under the system temporary folder and deleted; nothing was left in the repository.

#	Command (from `backend/` unless said)	Real output (key lines)
1	`time ../.venv/bin/python -B -m scripts.gap_report_events`	The table in section 7.4; `5.012 total`
2	`time ../.venv/bin/python -B -m scripts.gap_report_restaurant`	`OR-tools available: True`, the tables in section 7.7; `10.483 total`
3	Events gap report run again; first nine columns of both runs compared with `cmp`	`identical score columns`
4	`../.venv/bin/python -c "import ortools, sys; ..."`	`ortools 9.15.6755 python 3.12.13`
5	`../.venv/bin/python -c "import fastapi, sqlalchemy, pydantic; ..."`	`0.162.2 2.1.8 2.14.3`
6	`../.venv/bin/python -m pytest -p no:cacheprovider`	`443 passed, 1 warning in 31.24s`
7	`... -k "race or threads or concurrent"`	`4 passed, 439 deselected, 1 warning in 2.51s`
8	`... -m hypothesis`	`12 passed, 431 deselected, 1 warning in 2.25s`
9	`../.venv/bin/ruff check .`; `../.venv/bin/ruff format --check .`	`All checks passed!`; `135 files already formatted`
10	`cd frontend && npm test`	`Test Files 15 passed (15)`; `Tests 132 passed (132)` (133 in the final verification, which added one test)
11	`cd frontend && npm run typecheck`	exit code 0
12	Copy of `backend/` in a temporary folder, `uq_holds_resource` removed, `pytest tests/test_holds.py -k thirty`	`1 failed, 1 passed, 8 deselected, 1 warning in 0.84s`
13	Same copy, `uq_assignment_seat` also removed, same command	`2 failed, 8 deselected, 1 warning in 0.90s`
14	`DATABASE_URL=sqlite:///<temp file> DEMO_MODE=1 ../.venv/bin/python -m app.seed.run`, then counts with `sqlite3`	`Demo data ready.`; wedding 72 guests, 43 seated, 80 seats, 10 tables; summit 116 guests, 58 seated, 150 seats, 30 tables; Casa Lumen 18 tables; 325 bookings; 24 tables in the database
15	`find` and `wc -l` over `backend/app`, `backend/tests`, `frontend/src`	83 files, 11,917 lines; 41 test files, 6,889 lines; 122 files, 11,753 lines; CSS 3,190 lines (final verification recount: 122 files, 11,756 lines; CSS 3,196 lines)
16	Count of paths and operations in `docs/openapi.json`	`paths 56 ops 65`
17	After the S8 and S9 work: `docs/openapi.json` exported again (docs/api.md), then counted	`3.1.0 SeatPlan API 0.1.0 paths 56 ops 65 schemas 38`

#	Command (from `backend/` unless said)	Real output (key lines)
18	After the S8 and S9 work: `find backend/app backend/tests -name '*.py'` (no `__pycache__`), files and `wc -l`	`backend/app` 84 files, 12,107 lines; `backend/tests` 44 files, 7,475 lines